



TRUNG TÂM TIN HỌC
Trường Đại học Khoa Học Tự Nhiên - Đại học quốc gia TP.HCM

LẬP TRÌNH VIÊN CÔNG NGHỆ .NET

Module 2

Click vào phụ lục để chuyển tới bài cần đọc

Phụ lục

Bài 1 Tính thừa kế và đa hình của class	2
Bài 2 Tổng quản về SQL Server	15
Bài 3 Ngôn ngữ SQL	25
Bài 4 Tổng quan ADO.Net và Kết nối với CSDL	38
Bài 5 Mô hình kết nối	51
Bài 6 Mô hình ngắt kết nối	66
Bài 7 XML cơ bản	86
Bài 8 LINQ nâng cao.....	97
Bài 9 Entity Framework	107





Trường ĐH Khoa Học Tự Nhiên Tp. Hồ Chí Minh
TRUNG TÂM TIN HỌC

LTV CÔNG NGHỆ .NET

Module 2 - Bài 1. *Tính thừa kế và đa hình của class*

Ngành Lập trình & CSDL

<http://www.t3h.vn>

2014



2014



Nội dung

1. Tính thừa kế (Inheritance)

1. Khái niệm
2. Xây dựng lớp thừa kế

2. Tính đa hình (Polymorphism) của class

1. Khái niệm
2. Từ khóa this, base
3. Ghi đè (overriding)
4. Nạp chồng hàm (Overloading)
5. Kết nối trễ (Late binding)



LTV Công Nghệ .Net (2014) – Module2 - Bài 1. Tính thừa kế và đa hình của class

2

1. Tính thừa kế (Inheritance)

□ Khái niệm



LTV Công Nghệ .Net (2014) – Module2 - Bài 1. Tính thừa kế và đa hình của class

3



1. Tính thừa kế (Inheritance)

□ Khái niệm

- Tính thừa kế là một khái niệm nền tảng cho phép tái sử dụng mã lệnh đang tồn tại và điều này giúp tiết kiệm được thời gian trong việc lập trình
- Các class có thể thừa kế từ class khác. Class mới được gọi là **class dẫn xuất** (hay còn gọi là class con) sẽ được quyền truy xuất đến tất cả các thành viên dữ liệu và các phương thức không có sử dụng bảo từ **private** của **class cơ sở** (hay còn gọi là class cha)
- Ngầm định một lớp được tạo ra sẽ kế thừa lớp **System.Object**
- Một lớp chỉ được phép kế thừa tối đa 1 lớp, trừ khi được khai báo **sealed**
- C# cho phép kế thừa phân cấp (inheritance hierarchy)



LTV Công Nghệ .Net (2014) – Module2 - Bài 1. Tính thừa kế và đa hình của class

4

1. Tính thừa kế (Inheritance)



□ Xây dựng lớp thừa kế

- Cú pháp:
Tên_class_con : Tên_class_cơ_sở
- Ví dụ: xét class cơ sở khai báo như sau:

```
class LopCha
{
    public int giatri;
    public static string mThongtin = "";
    public LopCha()
    {
        mThongtin += "Phương thức khởi tạo của lớp cha \n";
        giatri=10;
    }
    public string Xuat()
    {
        return "Phương thức xuất của lớp cha: giá trị=" + giatri;
    }
}
```



LTV Công Nghệ .Net (2014) – Module2 - Bài 1. Tính thừa kế và đa hình của class

5



1. Tính thừa kế (Inheritance)

❑ Xây dựng lớp thừa kế

- Ví dụ: tạo class dẫn xuất từ class cơ sở với khai báo như sau:

```
class LopCon: LopCha
{
    public LopCon()
    {
        mThongtin += "Phương thức khởi tạo của lớp con";
    }
}

//Đoạn code sử dụng
private void button1_Click(object sender, EventArgs e)
{
    LopCon lc = new LopCon();
    lblKetqua.Text = LopCha.mThongtin;
    lblKetqua2.Text = lc.Xuat();
    lc.giatri = 20;
    lblKetqua3.Text = lc.Xuat();
}
```

// Kết quả xuất ra màn hình khi thực thi ví dụ trên:

Phương thức khởi tạo của lớp cha

Phương thức khởi tạo của lớp con

Phương thức xuất của lớp cha: giá trị=10

Phương thức xuất của lớp cha: giá trị=20



1. Tính thừa kế (Inheritance)



❑ Xây dựng lớp thừa kế

- Các từ khóa quy định phạm vi sử dụng các thành phần khai báo trong lớp cha:
 - Từ khóa **protected**: cho phép lớp con sử dụng
 - Từ khóa **private**: không cho phép lớp con sử dụng
 - Từ khóa **virtual**: cho phép lớp con ghi đè với từ khóa **override**
- Do đó:
 - Lớp dẫn xuất chỉ được phép kế thừa các thành phần được khai báo với từ truy xuất là public hoặc protected ở lớp cơ sở





1. Tính thừa kế (Inheritance)

❑ Xây dựng lớp thừa kế

- Ví dụ: bổ sung phương thức sau vào LopCha, cài đặt lại phương thức này vào lớp con

```
//Cài đặt phương thức vào LopCha
public virtual string phuongThucVirtual()
{
    return "Đây là phương thức Virtual của lớp cha";
}

//Cài đặt phương thức trên vào LopCon
public override string phuongThucVirtual()
{
    return base.phuongThucVirtual() + "\n Được lớp con sử dụng lại và ghi
    đề lên";
}

//từ khóa base dùng để tham chiếu đến thành phần của lớp cha
```



LTV Công Nghệ .Net (2014) – Module2 - Bài 1. Tính thừa kế và đa hình của class

8

1. Tính thừa kế (Inheritance)



❑ Xây dựng lớp thừa kế

- Ví dụ: bổ sung phương thức sau vào LopCha, cài đặt lại phương thức này vào lớp con → cài đặt code sử dụng

```
LopCon lc = new LopCon();
lblKetqua.Text = lc.phuongThucVirtual();

//kết quả xuất ra màn hình:
```

Đây là phương thức Virtual của lớp cha
Được lớp con sử dụng lại và ghi đề lên



LTV Công Nghệ .Net (2014) – Module2 - Bài 1. Tính thừa kế và đa hình của class

9



1. Tính thừa kế (Inheritance)

❑ Xây dựng lớp thừa kế

• Tóm tắt:

```
LopCha
{
    public phươngThuc_01
    protected phươngThuc_02
    public virtual phươngThuc_03
    private phươngThuc_04
}
```



```
LopCon: LopCha
{
    phươngThuc_01
    phươngThuc_02
    public override phươngThuc_03
    phươngThuc_04
}
```

Được phép sử dụng phương thức 01 và 02

Được phép tái định nghĩa lại phương thức 03



LTV Công Nghệ .Net (2014) – Module2 - Bài 1. Tính thừa kế và đa hình của class

10



Nội dung

1. Tính thừa kế (Inheritance)

1. Khái niệm
2. Xây dựng lớp thừa kế

2. Tính đa hình (Polymorphism) của class

1. Khái niệm
2. Từ khóa this, base
3. Ghi đè (overriding)
4. Nạp chồng hàm (Overloading)
5. Kết nối trễ (Late binding)



LTV Công Nghệ .Net (2014) – Module2 - Bài 1. Tính thừa kế và đa hình của class

11



2. Tính đa hình (Polymorphism)

❑ Khái niệm

- Là khả năng cho phép xử lý các đối tượng khác nhau trên cùng một phương thức
 - Hay nói một cách khác, là nó có khả năng gọi cùng một thông điệp đến những đối tượng khác nhau
 - Ví dụ: thông điệp “**Tính chu vi**” được gọi đến cả hai đối tượng hình chữ nhật và hình tròn. Trong hai đối tượng này đều có chung phương thức “**Chuvi**”, tuy nhiên tùy theo thời điểm mà đối tượng nhận thông điệp, lúc đó hình tương ứng sẽ được tính chu vi
- Tính đa hình được thể hiện dưới các hình thức:
 - Ghi đè (overriding)
 - Nạp chồng (Overloading)
 - Kết nối trễ (Late binding)



LTV Công Nghệ .Net (2014) – Module2 - Bài 1. Tính thừa kế và đa hình của class

12



2. Tính đa hình (Polymorphism)

❑ Sử dụng từ khóa this, base

- Từ khóa **base**:
 - Được sử dụng để tham chiếu đến lớp cơ sở từ lớp dẫn xuất
 - Ví dụ: Tạo lớp cha NhanVien

```
public class NhanVien
{
    string hoTen;
    int tuoi;
    public NhanVien()           // phương thức khởi tạo không tham số
    {
        hoTen="";
        tuoi=0;
    }
    public NhanVien(string pHoten, int pTuoi) // phương thức khởi tạo có tham số
    {
        hoTen=pHoten;
        tuoi=pTuoi;
    }
}
```



LTV Công Nghệ .Net (2014) – Module2 - Bài 1. Tính thừa kế và đa hình của class

13



2. Tính đa hình (Polymorphism)

❑ Sử dụng từ khóa this, base

• Từ khóa **base**:

- Được sử dụng để tham chiếu đến lớp cơ sở từ lớp dẫn xuất
- Ví dụ: Tạo lớp con NVVanPhong

```
public class NVVanPhong : NhanVien
{
    int soNgayVang;
    public NVVanPhong(string pHoten, int pTuoi, int pNgayVang):base(pHoten, pTuoi)
    {
        soNgayVang = pNgayVang;
    }
}
```

*// để tham chiếu đến phương thức khởi tạo có tham số trong lớp cơ sở NhanVien phải sử dụng từ khóa **base**. Nếu không khai báo thì xem như tham chiếu đến p.thức khởi tạo không tham số*



2. Tính đa hình (Polymorphism)

❑ Sử dụng từ khóa this, base

• Từ khóa **this**:

- Được sử dụng để tham chiếu đến lớp hiện hành (lớp chứa đoạn lệnh đang cài đặt)
- Ví dụ: Sửa lại Constructor của lớp con NVVanPhong

```
public class NVVanPhong : NhanVien
{
    int soNgayVang;
    public NVVanPhong(string pHoten, int pTuoi, int pNgayVang):base(pHoten, pTuoi)
    {
        this.soNgayVang = pNgayVang;
    }
}
```





2. Tính đa hình (Polymorphism)

❑ Tính ghi đè (overriding)

- Khái niệm ghi đè được dùng để định nghĩa lại phương thức của lớp cơ sở (lớp cha) trong lớp dẫn xuất (lớp con kế thừa)
- Các điểm cần lưu ý khi thực hiện ghi đè:
 - Phương thức ở lớp cơ sở và lớp dẫn xuất phải có cùng dạng hàm (signature) và kiểu dữ liệu trả về
 - Phương thức lớp cơ sở phải được khai báo với từ khóa **virtual**
 - Phương thức lớp dẫn xuất phải được khai báo với từ khóa **override**



2. Tính đa hình (Polymorphism)

❑ Tính ghi đè (overriding)

- Ví dụ: Bổ sung phương thức `tinhLuong()` vào `LopCha`, sau đó tạo phương thức ghi đè trong `LopCon`

```
//Bổ sung trong LopCha
public virtual float tinhLuong()
{
    return heSo * 1050000;
}

//Ghi đè trong LopCon
public override float tinhLuong()
{
    return base.tinhLuong() - soNgayVang * 50000;
}
```





2. Tính đa hình (Polymorphism)

❑ Nạp chồng hàm (overloading)

- Còn được gọi là nạp chồng phương thức
- Cho phép khai báo các phương thức trùng tên nhau nhưng có tham số khác nhau
- Các điểm cần lưu ý khi thực hiện nạp chồng hàm:
 - Tên của các phương thức phải trùng nhau
 - Số lượng tham số phải khác nhau
 - Kiểu dữ liệu của các tham số và thứ tự các tham số phải khác nhau

(Kỹ thuật này đã được trình bày trong Module 01)



2. Tính đa hình (Polymorphism)

❑ Kết nối trễ (Late binding)

- Tính đa hình dựa trên sự nối kết (Binding), đó là quá trình gắn một phương thức với một hàm thực sự
 - Trình biên dịch chưa thể xác định hàm nào tương ứng với phương thức nào sẽ được gọi mà chỉ xác định được vào lúc chương trình chạy → Điều này gọi là sự kết nối muộn (Late binding)



2. Tính đa hình (Polymorphism)

❑ Kết nối trễ (Late binding)

- Ví dụ: hãy quan sát bài toán đơn giản minh họa tính đa hình, giao diện màn hình như sau:

Tính lương cho nhân viên văn phòng

Tính lương cho nhân viên sản xuất



LTV Công Nghệ .Net (2014) – Module2 - Bài 1. Tính thừa kế và đa hình của class

20

2. Tính đa hình (Polymorphism)

❑ Kết nối trễ (Late binding)

- Tạo các class cơ sở NhanVien:

```
public class NhanVien
{
    string hoTen;
    int tuoi;
    float heSo;
    public NhanVien() { }
    public NhanVien(string pHoten, int pTuoi, float pHeso)
    {
        hoTen=pHoten;
        tuoi=pTuoi;
        heSo = pHeso;
    }
    public virtual float tinhLuong()
    {
        return heSo * 1050000;
    }
}
```



LTV Công Nghệ .Net (2014) – Module2 - Bài 1. Tính thừa kế và đa hình của class

21



2. Tính đa hình (Polymorphism)

❑ Kết nối trễ (Late binding)

- Tạo các 2 class dẫn xuất NVVanPhong và NVSanXuat

```
public class NVVanPhong : NhanVien
{
    int soNgayVang;
    public NVVanPhong(string pHoten, int pTuoi,
float pHeso, int pNgayVang):base(pHoten, pTuoi,
pHeso)
    {
        this.soNgayVang = pNgayVang;
    }
    public override float tinhLuong()
    {
        return base.tinhLuong() - soNgayVang *
50000;
    }
}
```

```
public class NVSanXuat : NhanVien
{
    int soSanPham;
    public NVSanXuat(string pHoten, int pTuoi,
float pHeso, int pSoLuong): base(pHoten, pTuoi,
pHeso)
    {
        this.soSanPham = pSoLuong;
    }
    public override float tinhLuong()
    {
        return base.tinhLuong() + soSanPham *
20000;
    }
}
```



2. Tính đa hình (Polymorphism)

❑ Kết nối trễ (Late binding)

- Code xử lý tính tiền lương:

```
private void btnTinhLuong_Click(object sender, EventArgs e)
{
    string hoTen = txtHoten.Text;
    int tuoi = Convert.ToInt32(txtTuoi.Text);
    float heso = Convert.ToSingle(txtHeso.Text);
    float tienluong; NhanVien nv;
    if (chkLaNVVP.Checked == true) {
        int ngayVang = Convert.ToInt32(txtSoNgayVang.Text);
        nv = new NVVanPhong(hoTen,tuoi,heso,ngayVang) ; //Late Binding
        tienluong = nv.tinhLuong(); //Công thức tính lương cho NV văn phòng
    }
    else {
        int soLuong = Convert.ToInt32(txtSoLuongSP.Text);
        nv = new NVSanXuat(hoTen,tuoi,heso,soLuong);
        tienluong = nv.tinhLuong(); //Công thức tính lương cho NV sản xuất
    }
    lblTienLuong.Text = string.Format("Tiền lương: {0:#,##0}", tienluong);
}
```



Thảo luận



LTV Công Nghệ .Net (2014) – Module2 - Bài 1. Tính thừa kế và đa hình của class

24



Trường Đại học Khoa Học Tự Nhiên Tp. Hồ Chí Minh
TRUNG TÂM TIN HỌC

LTV CÔNG NGHỆ .NET

Module 2 - Bài 2. *Tổng quan về SQL Server*

Ngành Lập trình & CSDL

<http://www.t3h.vn>

2014



2014





Nội dung

1. Các khái niệm cơ bản về SQL Server

1. Hệ quản trị cơ sở dữ liệu quan hệ
2. Cơ sở dữ liệu
3. Bảng
4. Cách xác thực khi đăng nhập

2. Tiện ích SQL Server Management Studio

1. Khởi động SSMS và đăng nhập vào SQL Server
2. Tạo cơ sở dữ liệu
3. Tạo bảng
4. Tạo Database Diagram
5. Nhập dữ liệu



LTV Công Nghệ .Net (2014) – Module2 - Bài 2. Tổng quan về SQL Server

2



1. Các khái niệm cơ bản về SQL Server

□ Hệ quản trị cơ sở dữ liệu (CSDL) quan hệ

- Relational Database Management System (RDBMS)
- Là hệ thống các chương trình hỗ trợ việc quản lý và khai thác dữ liệu theo mô hình cơ sở dữ liệu quan hệ
- Cài đặt xử lý theo mô hình Client – Server
- Cho phép đồng thời nhiều người dùng cùng khai thác dữ liệu
- Cho phép sao lưu và phục hồi lại dữ liệu khi gặp sự cố
- ...



LTV Công Nghệ .Net (2014) – Module2 - Bài 2. Tổng quan về SQL Server

3



1. Các khái niệm cơ bản về SQL Server

❑ Cơ sở dữ liệu – Database

- Là tập hợp các thông tin cùng ý nghĩa trong một tổ chức

❑ Quan hệ (Relation) là gì?

- Là một bảng gồm các cột (Column) và các dòng (Row)
- Các cột còn được gọi là trường (Field)
- Các dòng còn được gọi là mẫu tin (Record)

SINH VIÊN						
Mã	Họ	Tên	Ngày sinh	Giới tính	Học bổng	Mã khoa
C00	Nguyễn Thị	Trang	13/8/1991	No	100000	CN
C01	Hà	Tuấn	24/2/1991	Yes	0	CN
C02	Trần Ngọc	Hoà	11/6/1990	No	120000	CN
C03	Bùi Thị	Thảo	26/2/1990	No	100000	CN
C04	Nguyễn Hoàng	Hùng	19/3/1990	Yes	150000	CN
T00	Lê	Tuấn	15/2/1991	Yes	0	TO
T01	Bùi Minh	Khánh	9/4/1990	Yes	120000	TO
T02	Trần Thị	Lan	4/3/1990	No	100000	TO
T03	Lê	Thiện	18/5/1990	Yes	0	TO
T04	Lê Thị	Thảo	27/3/1990	No	120000	TO



1. Các khái niệm cơ bản về SQL Server

❑ Bảng – Table

- Bảng dùng để lưu trữ các thông tin của một đối tượng trong thế giới thực
- Bảng có cấu trúc gồm các dòng (Row) và các cột (Column)
- Bảng trong CSDL thường có khoá chính (Primary key)
 - Khóá chính không cho phép chứa giá trị null và trùng nhau
- Một table có thể có hoặc không có khoá ngoại (Foreign Key)
 - Khóá ngoại được phép null, chứa giá trị phải tồn tại bên table khác
- Các bảng thường liên hệ với nhau bằng các mối quan hệ
- Bảng được tạo trong các Schema (mặc định là Schema dbo)





1. Các khái niệm cơ bản về SQL Server

❑ Cách xác thực khi đăng nhập – Authentication

- Windows Authentication: xác thực các user được cấp trên máy
- SQL Server Authentication: xác thực các user được cấp trong SQL Server



Nội dung



1. Các khái niệm cơ bản về SQL Server

1. Hệ quản trị cơ sở dữ liệu quan hệ
2. Cơ sở dữ liệu
3. Bảng
4. Cách xác thực khi đăng nhập

2. Tiềm ích SQL Server Management Studio

1. Khởi động SSMS và đăng nhập vào SQL Server
2. Tạo cơ sở dữ liệu
3. Tạo bảng
4. Tạo Database Diagram
5. Nhập dữ liệu



2. Tiện ích SQL Server Management Studio

❑ Khởi động SSMS và đăng nhập vào SQL Server

- Khởi động SSMS
 - Thực đơn Programs → Microsoft SQL Server → SQL Server Management Studio



LTV Công Nghệ .Net (2014) – Module2 - Bài 2. Tổng quan về SQL Server

8

2. Tiện ích SQL Server Management Studio

❑ Khởi động SSMS và đăng nhập vào SQL Server

- Đăng nhập vào SQL Server
 - **Server type:** Database Engine
 - **Server name:** khai báo server, các giá trị có thể sử dụng:
 - Dấu chấm ".", (**local**) : Máy cục bộ
 - Tên server hoặc địa chỉ IP: Máy cục bộ hoặc máy chủ
 - Nếu dùng SQL Server Express Edition: thêm **\SQLEXPRESS** sau các giá trị trên
 - **Authentication:** Windows hoặc SQL Server
 - Nếu chọn SQL Server Authentication thì nhập tiếp: User name và Password



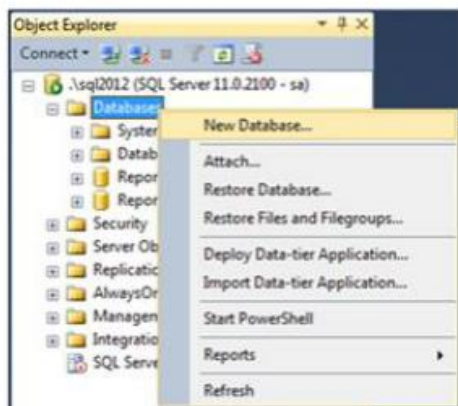
LTV Công Nghệ .Net (2014) – Module2 - Bài 2. Tổng quan về SQL Server

9

2. Tiện ích SQL Server Management Studio

❑ Tạo cơ sở dữ liệu – Database

- Nhấn nút phải chuột tại mục **Databases**
- Chọn **New Database ...**
- Nhập tên CSDL và nhấn OK



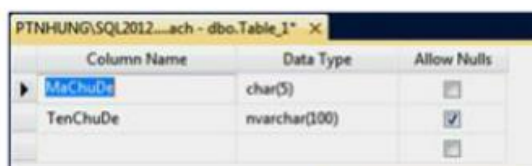
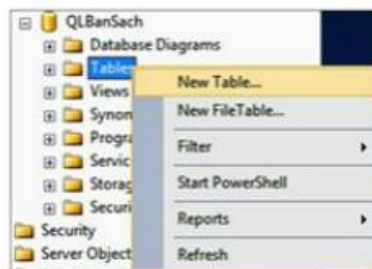
LTV Công Nghệ .Net (2014) – Module2 - Bài 2. Tổng quan về SQL Server

10

2. Tiện ích SQL Server Management Studio

❑ Tạo bảng – Table

- Chọn Database muốn tạo Table
- Nhấn nút phải chuột tại mục **Tables**
- Chọn **New Table ...**
- Lần lượt tạo các field
- Tạo khóa chính:
 - Chọn field, click phải chuột,
 - Chọn **Set Primary Key**
- Lưu lại và đặt tên Table:
 - Nhấn **Ctrl + S**



LTV Công Nghệ .Net (2014) – Module2 - Bài 2. Tổng quan về SQL Server

11



2. Tiện ích SQL Server Management Studio

❑ Tạo bảng – Table

- Một số kiểu dữ liệu cơ bản
 - Kiểu chuỗi: Char, Varchar, NChar, NVarchar
 - Kiểu số: Int, Real, Money
 - Kiểu ngày: DateTime
 - Kiểu luận lý: Bit



2. Tiện ích SQL Server Management Studio

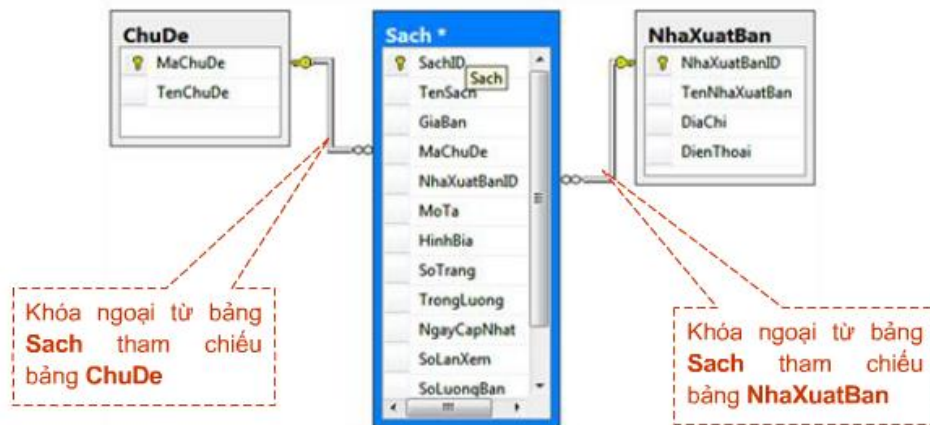
❑ Tạo Database Diagram

- Để khai báo các ràng buộc khóa ngoại
- Thao tác:
 - Chọn Database muốn tạo Diagram
 - Nhấn nút phải chuột tại mục **Database Diagrams**
 - Chọn **New Database Diagram**
 - Đưa các Table vào Diagram
 - Kéo và thả các cột từ bảng cha đến bảng con
 - Lưu Diagram



2. Tiện ích SQL Server Management Studio

❑ Tạo Database Diagram



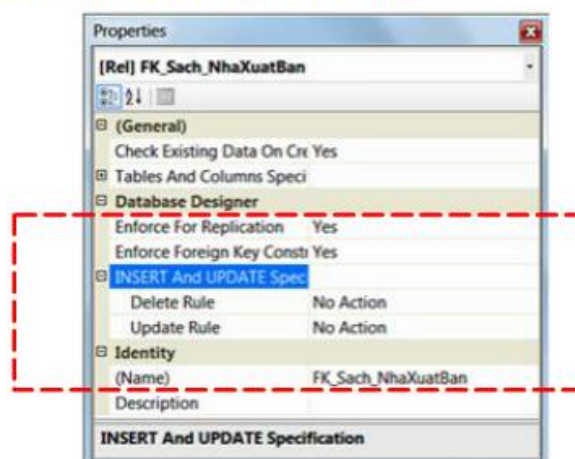
LTV Công Nghệ .Net (2014) – Module2 - Bài 2. Tổng quan về SQL Server

14

2. Tiện ích SQL Server Management Studio

❑ Tạo Database Diagram

- Các thuộc tính cơ bản của quan hệ
 - **Thao tác mở cửa sổ Properties:** Chọn dây quan hệ → click phải chuột vào và chọn chức năng **Properties**.



LTV Công Nghệ .Net (2014) – Module2 - Bài 2. Tổng quan về SQL Server

15

2. Tiện ích SQL Server Management Studio

❑ Tạo Database Diagram

- Các thuộc tính cơ bản của quan hệ
 - Enforce Foreign Key Constraints: Bắt buộc kiểm tra khóa ngoại
 - Delete Rule:
 - No Action: Không cho phép xóa nếu có dữ liệu quan hệ bên bảng con
 - Cascade: Xóa dữ liệu có quan hệ bên bảng con
 - Update Rule:
 - No Action: Không cho phép cập nhật nếu có dữ liệu quan hệ bên bảng con
 - Cascade: Cập nhật dữ liệu có quan hệ bên bảng con
 - Identity (Name): Tên của quan hệ



2. Tiện ích SQL Server Management Studio

❑ Tạo Database Diagram

- Ví dụ:

NhaXuatBan

NhaXuatBanID	TenNhaXuatBan
1	Nhà xuất bản Trẻ
2	NXB Thống kê
3	Kim đồng
4	Đại học quốc gia
5	Văn hóa nghệ thuật
6	Văn hóa
7	NXB Lao động - Xã hội
8	Khoa Học & Kỹ Thuật
9	Thanh Niên

Sach

SachID	TenSach	NhaXuatBanID
1	Tư Học Photoshop CS5 Qua Hình Ảnh Minh Họa (Sách K...	1
2	Sketchup & Vray Trong Thiết Kế Kiến Trúc	1
3	Essential Words For The IELTS (Kèm 1 đĩa CD)	1
4	Công Nghệ Mạng Máy Tính	1
5	36 71 Kỹ Thuật Thẩm Nhập Windows Vista Pro	2
6	41 Tạo Bản Thuyết Trình Trong Microsoft Office PowerPoint...	2
7	48 Tinh Thúc	2
8	45 Nhập Môn Windows Vista Tập 1	2
9	27 Công Chúa Aurora Và Chú Rồng Nhỏ	3
10	3 Tư Học InDesign CS3	3
11	11 Nghệ Thuật Sống An Lạc	3
12	15 Lập Bản Đồ Tư Duy - Công Cụ Tư Duy Tối Ưu Sẽ Làm T...	3
13	21 17 Nguyên Tắc Vàng Trong Làm Việc Nhóm	3
14	4 Đặc Nhân Tâm - Cuốn Sách Hay Nhất Của Mọi Thời Đại ...	4
15	38 SQL Server 2005 - Xây Dựng Ứng Dụng Quản Lý Kế To...	4



2. Tiện ích SQL Server Management Studio



- ❑ Nhập dữ liệu
 - Chọn Table, nhấn nút phải chuột
 - Chọn **Edit Top 200 rows**



Thảo luận





Trường ĐH Khoa Học Tự Nhiên Tp. Hồ Chí Minh
TRUNG TÂM TIN HỌC

LTV CÔNG NGHỆ .NET

Module 2 - Bài 3. *Ngôn ngữ SQL*

Ngành Lập trình & CSDL

<http://www.t3h.vn>

2014



2014





Nội dung

1. Tổng quan về ngôn ngữ SQL

1. Giới thiệu ngôn ngữ SQL
2. Phân loại ngôn ngữ

2. Truy vấn dữ liệu

3. Cập nhật dữ liệu



LTV Công Nghệ .Net (2014) – Module2 - Bài 3. Ngôn ngữ SQL

2

1. Tổng quan về ngôn ngữ SQL



□ Giới thiệu ngôn ngữ SQL

- Structured Query Language (ngôn ngữ truy vấn có cấu trúc)
- Được công ty máy tính IBM đề xuất vào năm 1970
- Ngôn ngữ dùng để truy vấn dữ liệu (dạng tiếng Anh)
- Ví dụ:

```
SELECT SachID, TenSach, MoTa  
FROM Sach
```

□ Phân loại ngôn ngữ SQL

- Ngôn ngữ định nghĩa dữ liệu (Data Definition Language)
- Ngôn ngữ **truy vấn** dữ liệu (Structured Query Language)
- Ngôn ngữ **thao tác** dữ liệu (Data Manipulation Language)
- Ngôn ngữ điều khiển dữ liệu (Data Control Language)



LTV Công Nghệ .Net (2014) – Module2 - Bài 3. Ngôn ngữ SQL

3



Nội dung

1. Tổng quan về ngôn ngữ SQL
2. Truy vấn dữ liệu
 1. Truy vấn đơn giản trên một bảng
 2. Truy vấn đơn giản trên nhiều bảng
 3. Truy vấn có nhóm
3. Cập nhật dữ liệu



2. Truy vấn dữ liệu



☐ Truy vấn đơn giản trên một bảng

• Cú pháp 1:

```
Select <Cột>|<Biểu_thức>  
From Tên bảng  
[Order By <Cột_sắp_thứ_tự>]
```

▪ Ví dụ 1:

```
Select *  
From Sach
```

▪ Ví dụ 2:

```
Select SachID, TenSach, MoTa  
From Sach
```





2. Truy vấn dữ liệu

❑ Truy vấn đơn giản trên một bảng

• Cú pháp 2: lọc dữ liệu

```
Select <Cột>|<Biểu thức>
From Tên bảng
[Where <biểu thức lọc>]
[Order By <Cột sắp thứ tự>]
```

▪ Ví dụ:

```
Select *
From Sach
Where GiaBan > 150000
```



2. Truy vấn dữ liệu

❑ Truy vấn đơn giản trên một bảng

• Cú pháp 2:

▪ Với <Biểu thức lọc> có chứa các toán tử:

- Các toán tử luận lý: And, Or, Not
- Các toán tử so sánh: >, >=, <, <=, =, <>, Like
- Từ khóa **Between ... And**: kiểm tra trong một vùng
GiaBan Between 100000 And 200000
- Từ khóa **In**: kiểm tra trong một tập hợp
MaChuDe In ('LUAT', 'CNTTI', 'AMNHA')





2. Truy vấn dữ liệu

❑ Truy vấn đơn giản trên một bảng

• Cú pháp 2:

- Ví dụ 1: liệt kê các sách có đơn giá lớn hơn 150 ngàn và nhà xuất bản là số 1

```
Select *
From Sach
Where GiaBan > 150000 And NhaXuatBanID =1
```

- Ví dụ 2: liệt kê các sách có Tên sách bắt đầu bằng chữ T

```
Select * From Sach Where TenSach Like N'T%'
//Liệt kê các sách có tên bắt đầu bằng N và chiều dài tên là 4 ký tự
Select * From Sach Where TenSach Like N'N____'
```



2. Truy vấn dữ liệu

❑ Truy vấn đơn giản trên một bảng

• Cú pháp 2:

- Ví dụ 3: liệt kê các sách có đơn giá trong khoảng từ 100 ngàn đến 200 ngàn

```
Select *
From Sach
Where GiaBan Between 100000 And 200000
```

- Ví dụ 4: liệt kê các sách có Mã Chủ đề là 'CNTTI', 'LUAT' và 'DULIC'

```
Select * From Sach
Where MaChuDe IN ('CNTTI', 'LUAT', 'DULIC')
```





2. Truy vấn dữ liệu

❑ Truy vấn đơn giản trên nhiều bảng

• Cú pháp 3:

```
Select <Cột>|<Biểu thức>
From <Bảng 1> Liên_kết <Bảng 2>
[Where <Điều kiện lọc>]
[Order By <Cột sắp thứ tự>]
```

• Với Liên_kết là :

- **Inner Join** (bảng): Chỉ liên kết khi các mẫu tin của hai bảng có giá trị trên cột liên kết là bằng nhau, cú pháp như sau:

```
<Bảng 1> Inner Join <Bảng 2>
On <Bảng 1>.Cột= <Bảng 2>.Cột
```

▪ Ví dụ:

```
Sach Inner Join ChuDe
On Sach.MaChuDe = ChuDe.MaChuDe
```



2. Truy vấn dữ liệu



❑ Truy vấn đơn giản trên nhiều bảng

• Với Liên_kết là :

▪ **Left Join** (ưu tiên bên trái) :

- Liên kết khi các mẫu tin của hai bảng có giá trị trên cột liên kết là bằng nhau. Và
- “Ưu tiên” cho các mẫu tin của bảng bên trái
- Cú pháp:

```
<Bảng 1> Left Join <Bảng 2>
On <Bảng 1>.Cột= <Bảng 2>.Cột
```

▪ Ví dụ:

```
Sach Left Join ChuDe
On Sach.MaChuDe = ChuDe.MaChuDe
```





2. Truy vấn dữ liệu

❑ Truy vấn đơn giản trên nhiều bảng

• Với Liên_kết là :

▪ Right Join (ưu tiên bên phải) :

- Liên kết khi các mẫu tin của hai bảng có giá trị trên cột liên kết là bằng nhau. Và
- “Ưu tiên” cho các mẫu tin của bảng bên phải
- Cú pháp:

```
<Bảng 1> Right Join <Bảng 2>
On <Bảng 1>.Cột= <Bảng 2>.Cột
```

▪ Ví dụ:

```
Sach Right Join ChuDe
On Sach.MaChuDe = ChuDe.MaChuDe
```



2. Truy vấn dữ liệu



❑ Truy vấn có nhóm

• Cú pháp 4:

```
Select <Cột>|<Biểu thức>
From <Bảng 1> Liên kết <Bảng 2>
[Where <Điều kiện lọc cho mẫu tin>]
[Group By <Cột nhóm>]
[Having <Điều kiện lọc cho nhóm>]
[Order By <Cột sắp thứ tự>]
```

• Ví dụ 1: xem tổng số sách của từng chủ đề

```
Select MaChuDe, Count(*) As dem
From Sach
Group By MaChuDe
```





2. Truy vấn dữ liệu

❑ Truy vấn có nhóm

- Ví dụ 2: giống ví dụ 1 nhưng lọc dữ liệu theo nhóm; chỉ thể hiện các chủ đề nào có số sách lớn hơn 10

```
Select MaChuDe, Count(*) As dem
From Sach
Group By MaChuDe
Having Count(*) > 10
```

- Ví dụ 3: giống ví dụ 2 nhưng sắp xếp dữ liệu giảm dần theo cột Dem

```
Select MaChuDe, Count(*) As dem
From Sach Where GiaBan > 100000
Group By MaChuDe Having Count(*) > 10
Order By Count(*) Desc
```



2. Truy vấn dữ liệu

❑ Truy vấn có nhóm

- Ví dụ 4: giống ví dụ 2 nhưng bổ sung cột TenChuDe

```
Select s.MaChuDe, TenChuDe, Count(*) As dem
From Sach s Inner Join ChuDe cd ON s.MaChuDe = cd.MaChuDe
Group By s.MaChuDe, TenChuDe
```

- Trong ví dụ trên có sử dụng bí danh (alias) để thay thế tên Table, mục đích để viết ngắn gọn hơn
- Field MaChuDe có tên giống nhau trên 2 bảng Sach và ChuDe, do đó khi sử dụng ta phải khai báo theo cú pháp:

<Tên_Table> | <Bí_danh>.<Tên_field>





2. Truy vấn dữ liệu

❑ Truy vấn có nhóm

• Một số hàm thống kê:

- Đếm: **COUNT(*)**, **COUNT(Biểu thức)**
- Tính Tổng: **SUM(Biểu thức)**
- Tính Trung bình: **AVG(Biểu thức)**
- Tìm Giá trị nhỏ nhất: **MIN(Biểu thức)**
- Tìm Giá trị lớn nhất: **MAX(Biểu thức)**



Nội dung



1. Tổng quan về ngôn ngữ SQL

2. Truy vấn dữ liệu

3. Cập nhật dữ liệu

1. Thêm dòng
2. Sửa dữ liệu
3. Xóa dòng





3. Cập nhật dữ liệu

❑ Thêm dòng

- **Cú pháp 1:** Thêm 1 mẫu tin

```
Insert Into <Tên bảng> [(<Danh sách cột>)]
Values(<Danh sách giá trị>)
```

- Ví dụ 1:

```
Insert Into ChuDe (MaChuDe, TenChuDe)
Values('CNTTI', N'Công nghệ thông tin')
```

```
Insert Into ChuDe
Values('CNTTI', N'Công nghệ thông tin')
```



3. Cập nhật dữ liệu



❑ Thêm dòng

- **Cú pháp 2:** Thêm nhiều mẫu tin

```
Insert Into <Tên bảng> [(<Danh sách cột>)]
Select <Danh sách cột | biểu thức> From ...
```

- Ví dụ 2: thêm các mẫu tin từ table ChuDe vào table ChuDe_Copy

```
Insert Into ChuDe_Copy (MaChuDe, TenChuDe)
Select MaChuDe, TenChuDe
From ChuDe
```





3. Cập nhật dữ liệu

❑ Thêm dòng

- Lưu ý về kiểu dữ liệu khi thêm vào:
 - Dữ liệu kiểu chuỗi: Đặt trong nháy đơn
 - Ví dụ: 'Trời xanh mây trắng'
 - Dữ liệu kiểu ngày: Đặt trong nháy đơn
 - Ví dụ: '12/24/2008'
 - Dữ liệu kiểu số: 123456.789
 - Dữ liệu kiểu luận lý: 1 hoặc 0



3. Cập nhật dữ liệu



❑ Sửa dữ liệu

- **Cú pháp 1:** Cập nhật dữ liệu, không có liên quan đến table khác

Update <Tên bảng>
Set Cột1 = Giá trị1 [, Cột2 = Giá trị2, ...]
[Where <Điều kiện>]

- Ví dụ 1: tăng giá bán lên 10% đối với sách Công nghệ thông tin (điều kiện theo mã)

```
Update Sach
Set GiaBan = GiaBan * 1.1
Where MaChuDe = 'CNTTI'
```





3. Cập nhật dữ liệu

❑ Sửa dữ liệu

- **Cú pháp 2:** Cập nhật dữ liệu, có liên quan đến table khác

```
Update <Tên bảng>
Set Cột1 = Giá trị1 [, Cột2 = Giá trị2, ... ]
From <Bảng 1> Liên kết <Bảng 2>
[Where <Điều kiện>]
```

- Ví dụ 2: tăng giá bán lên 10% đối với sách Công nghệ thông tin

```
Update Sach
Set GiaBan = GiaBan * 1.1
From Sach s Inner Join ChuDe cd On s.MaChuDe = cd.MaChuDe
Where TenChuDe = N'Công nghệ thông tin'
```



3. Cập nhật dữ liệu



❑ Xóa dòng

- **Cú pháp 1:** Xóa dòng, không có liên quan đến table khác

```
Delete
From <Tên bảng>
[Where <Điều kiện>]
```

- Ví dụ 1: xóa các sách chưa khai báo giá bán

```
Delete
From Sach
Where GiaBan is null
```





3. Cập nhật dữ liệu

❑ Xóa dòng

- **Cú pháp 2:** Xóa dòng, có liên quan đến table khác

```
Delete <Tên bảng>
From <Bảng 1> Liên kết <Bảng 2>
[Where <Điều kiện>]
```

- Ví dụ 2: xóa các dòng trong table ThamGia có MaChuDe là 'LUAT'

```
Delete ThamGia
From ThamGia tg Inner Join Sach s On tg.SachID=s.SachID
Where MaChuDe='LUAT'
```



Thảo luận





Trường ĐH Khoa Học Tự Nhiên Tp. Hồ Chí Minh
TRUNG TÂM TIN HỌC

Go Screen Capture

LTV CÔNG NGHỆ .NET

Module 2 - Bài 4. *Tổng quan ADO.Net* và *Kết nối với CSDL*

Ngành Lập trình & CSDL

<http://www.t3h.vn>

2014



2014





Nội dung

1. Tổng quan về ADO.Net
 1. Khái niệm
 2. .Net Framework Data Providers
2. Mô hình truy xuất dữ liệu
3. Xử lý kết nối đến CSDL SQL Server
4. Minh họa kết nối với SQL Server



1. Tổng quan về ADO.Net

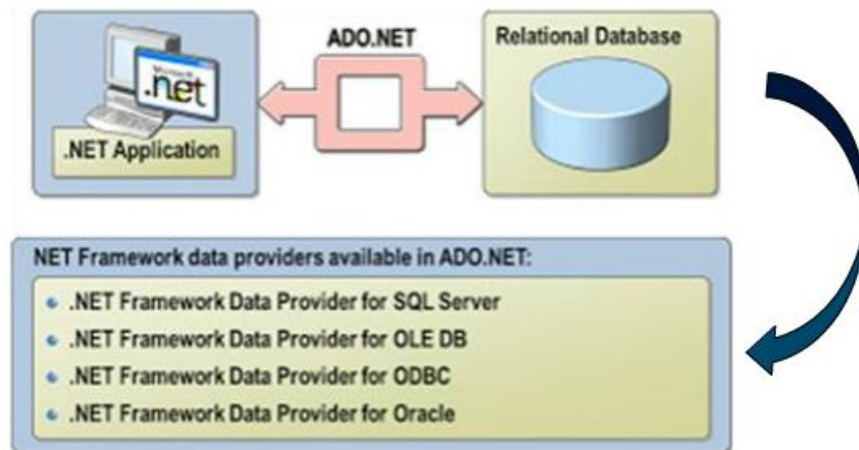
□ Khái niệm

- Là mô hình cho phép người lập trình thao tác với nhiều loại cơ sở dữ liệu theo cùng một cách, CSDL có thể là SQL Server, MS Access, Oracle, ...
- Ứng với mỗi loại CSDL sẽ là các **.NET Framework Data Provider** thích hợp, nghĩa là nó sẽ cung cấp các lớp đối tượng được sử dụng để thao tác với CSDL tương ứng
- ADO.NET cung cấp khả năng truy xuất và xử lý dữ liệu lớn, đồng thời trên hệ thống ứng dụng phân tán nhiều người dùng



1. Tổng quan về ADO.Net

□ Khái niệm



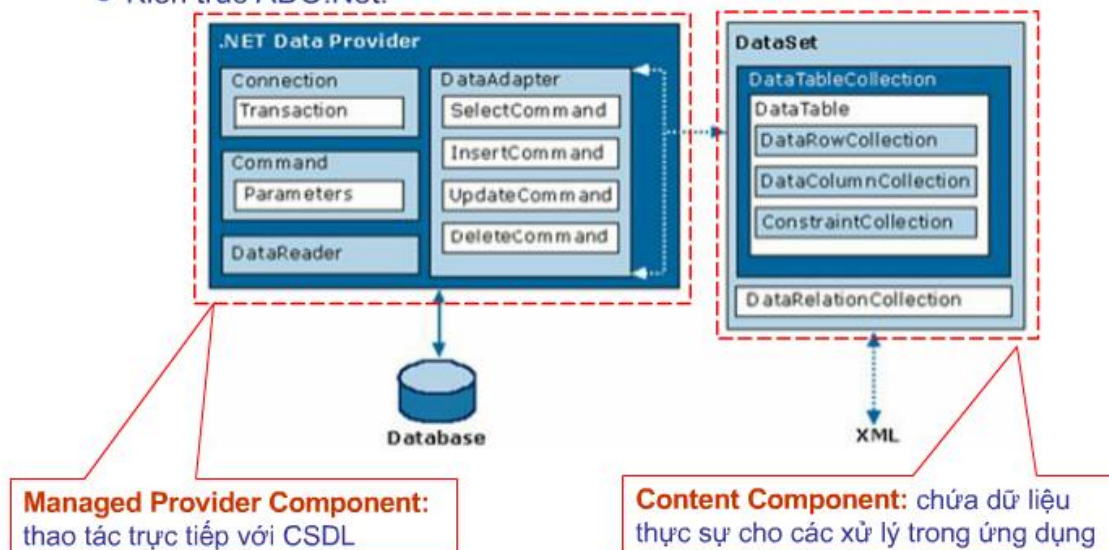
LTV Công Nghệ .Net (2014) – Module2 - Bài 4. Tổng quan ADO.Net và Kết nối CSDL

4

1. Tổng quan về ADO.Net

□ Khái niệm

• Kiến trúc ADO.Net:



LTV Công Nghệ .Net (2014) – Module2 - Bài 4. Tổng quan ADO.Net và Kết nối CSDL

5



1. Tổng quan về ADO.Net

□ Khái niệm

- Kiến trúc ADO.Net:
 - **Connection**: Dùng để tạo một kết nối (connection) đến CSDL
 - **Command**: Dùng để thực hiện câu lệnh, có thể truyền tham số, có thể nhận kết quả trả về
 - **DataReader**: Dùng để đọc dữ liệu theo cách forward-only và read-only
 - **DataAdapter**: Dùng để đọc dữ liệu từ CSDL và lưu trong DataSet
 - DataSet là một CSDL thu nhỏ được đặt trong bộ nhớ của máy client
 - Người dùng có thể thao tác trên DataSet và ghi lại các thay đổi trở lại CSDL
 - Dữ liệu đọc và lưu trong DataSet cũng có thể là dữ liệu XML



1. Tổng quan về ADO.Net

□ .Net Framework Data Providers

- **.NET Framework Data Provider for Microsoft SQL Server:**
 - Tối ưu khi kết nối với SQL Server (từ phiên bản 7.0)
 - Kết nối trực tiếp không thông qua ODBC hoặc OLE DB
 - Các lớp cung cấp thuộc **System.Data.SqlClient**
- **.NET Framework Data Provider for OLE DB**
 - Sử dụng native OLE DB
 - SQLOLEDB provider
 - MSDAORA provider
 - Microsoft.Jet.OLEDB.4.0 provider
 - Các lớp cung cấp thuộc **System.Data.OleDb**





1. Tổng quan về ADO.Net

□ .Net Framework Data Providers

● .NET Framework Data Provider for ODBC

- Sử dụng native ODBC Driver Manager
- SQL Server driver
- Microsoft ODBC for Oracle driver
- Microsoft Access driver
- Các lớp cung cấp thuộc **System.Data.Odbc**

● .NET Framework Data Provider for Oracle

- Kết nối Oracle Data Source thông qua Oracle client connectivity software (từ phiên bản 8.1.7)
- SQL Server driver
- Microsoft ODBC for Oracle driver, Microsoft Access driver
- Các lớp cung cấp thuộc **System.Data.OracleClient**



LTV Công Nghệ .Net (2014) – Module2 - Bài 4. Tổng quan ADO.Net và Kết nối CSDL

8



Nội dung

1. Tổng quan về ADO.Net

2. Mô hình truy xuất dữ liệu

1. Kết nối
2. Ngắt kết nối

3. Xử lý kết nối đến CSDL SQL Server

4. Minh họa kết nối với SQL Server



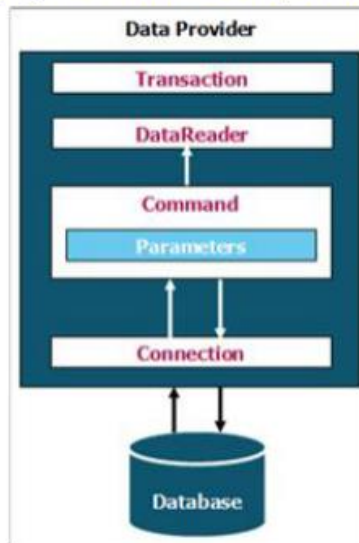
LTV Công Nghệ .Net (2014) – Module2 - Bài 4. Tổng quan ADO.Net và Kết nối CSDL

9

2. Mô hình truy xuất dữ liệu

❑ Mô hình kết nối - Connected Data Access

- Cần duy trì một kết nối mở trong suốt quá trình thao tác dữ liệu
- Sử dụng các đối tượng như **Connection, Command, DataReader**



LTV Công Nghệ .Net (2014) – Module2 - Bài 4. Tổng quan ADO.Net và Kết nối CSDL

10

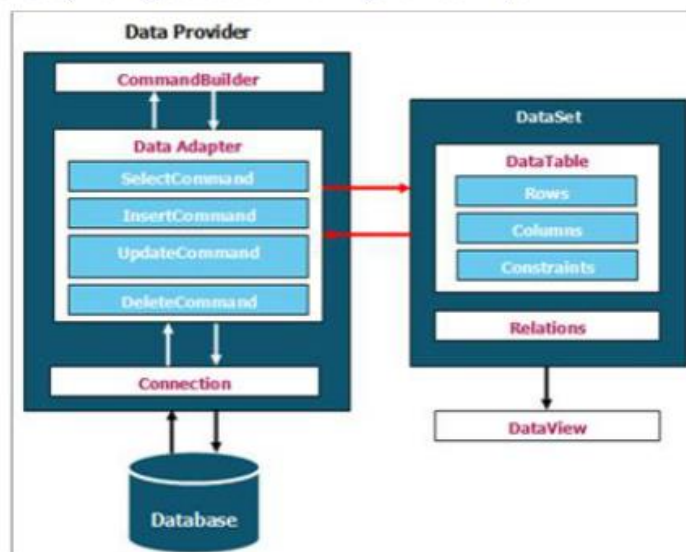
2. Mô hình truy xuất dữ liệu

❑ Mô hình ngắt kết nối - Disconnected Data Access

- Cho phép người dùng kết nối đến CSDL, đọc và lưu dữ liệu vào vùng nhớ của ứng dụng (DataSet) và ngắt kết nối với nguồn dữ liệu

- Sử dụng đối tượng

**Connection,
DataAdapter,
và DataSet**



LTV Công Nghệ .Net (2014) – Module2 - Bài 4. Tổng quan ADO.Net và Kết nối CSDL

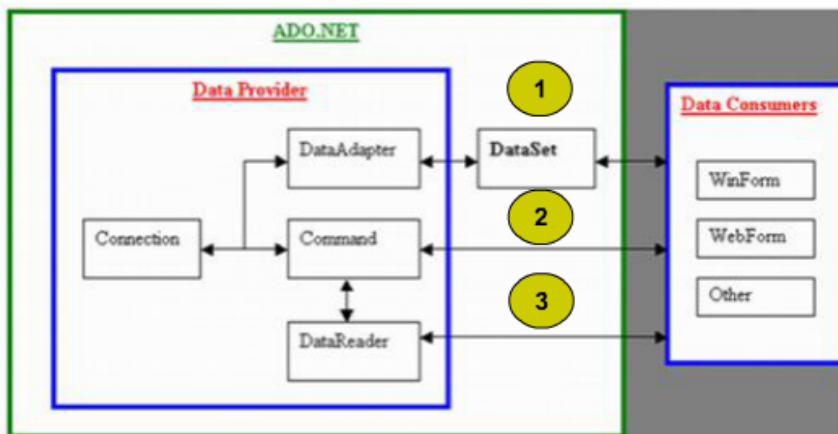
11



2. Mô hình truy xuất dữ liệu

❑ Tóm tắt các phương pháp truy xuất dữ liệu theo 2 mô hình trên:

- Có 3 phương pháp



LTV Công Nghệ .Net (2014) – Module2 - Bài 4. Tổng quan ADO.Net và Kết nối CSDL

12

Nội dung



1. Tổng quan về ADO.Net
2. Mô hình truy xuất dữ liệu
3. Xử lý kết nối đến CSDL SQL Server
 1. Tạo đối tượng Connection với việc sử dụng Connection String
 2. Mở đối tượng Connection
 3. Đóng đối tượng Connection
 4. Các thuộc tính của Connection
 5. Sự kiện StateChange
 6. Khai báo và sử dụng chuỗi kết nối trong tập cấu hình
4. Minh họa kết nối với SQL Server



LTV Công Nghệ .Net (2014) – Module2 - Bài 4. Tổng quan ADO.Net và Kết nối CSDL

13



3. Xử lý kết nối đến CSDL

❑ Tạo đối tượng Connection với việc sử dụng Connection String

- Thông tin chuỗi kết nối
 - **Tên Provider** tương ứng với CSDL
 - **Tên Server:** (local), . hoặc <địa chỉ IP> hoặc <tên máy>. Nếu SQL Server là bản Express thì thêm “\SQLEXPRESS”
 - **Tên Database**
 - **Kiểu đăng nhập** (Windows hoặc SQL Server):
 - **Windows:** **Integrated Security=SSPI (true)**
 - **SQL Server:**
 - **User ID = Tên_user**
 - **Password = Mật_khẩu**



3. Xử lý kết nối đến CSDL

❑ Tạo đối tượng Connection với việc sử dụng Connection String

- Thông tin chuỗi kết nối
 - Ví dụ:

Ví dụ 1: Tạo chuỗi kết nối đến CSDL Access (OleDb Data Provider)

```
string chuỗiKetNoi = @"Provider=Microsoft.Jet.OLEDB.4.0;Data Source = ..\..\DuLieu\QLSinhVien.mdb";
```

Ví dụ 2: Tạo chuỗi kết nối đến CSDL Excel (Odbc Data Provider)

```
string chuỗiKetNoi = @"Dsn=Excel Files; Dbq = ..\..\DuLieu\SINH_VIEN.xls";
```

Ví dụ 3: Tạo chuỗi kết nối đến CSDL SQL Server, bản Express (Sql Server Data Provider), chứng thực Windows

```
string chuỗiKetNoi = @"Server=.\SqlExpress; Database=QLBanSach; Integrated Security=SSPI";
```





3. Xử lý kết nối đến CSDL

- ❑ Tạo đối tượng Connection với việc sử dụng Connection String
 - Khai báo và khởi tạo

//Khai báo namespace hỗ trợ provider tương ứng

using System.Data.SqlClient;

//Tạo chuỗi thông tin kết nối đến QLBanSach

string chuoi = "Server=(local); Database=QLBansach; Integrated Security=SSPI";

//Tạo đối tượng kết nối - Connection

SqlConnection ketNoi = new SqlConnection(chuoi);



3. Xử lý kết nối đến CSDL

- ❑ Mở đối tượng Connection
 - Sử dụng phương thức **Open()**

đối_tượng_Connection.Open();

- ❑ Đóng đối tượng Connection
 - Sử dụng phương thức **Close()**

đối_tượng_Connection.Close();





3. Xử lý kết nối đến CSDL

❑ Các thuộc tính của Connection

- **State**: Cho biết trạng thái của kết nối
 - **ConnectionState.Closed**: Kết nối đã bị đóng
 - **ConnectionState.Open**: Kết nối đang mở
- **ConnectionString**: chuỗi kết nối
- **Database**: Tên của CSDL
- **DataSource**: Tên Server hoặc tên file CSDL

❑ Sự kiện StateChange

- Xảy ra khi kết nối thay đổi trạng thái.
- Hai giá trị của tham số e cho biết trạng thái:
 - **OriginalState**: Cho biết trạng thái trước khi thay đổi
 - **CurrentState**: Cho biết trạng thái hiện hành



LTV Công Nghệ .Net (2014) – Module2 - Bài 4. Tổng quan ADO.Net và Kết nối CSDL

18

3. Xử lý kết nối đến CSDL



❑ Ví dụ hoàn chỉnh:

```
using (SqlConnection ketNoi = new SqlConnection("Server=.; Database=QLBansach; Integrated
Security=SSPI"))
{
    try
    {
        ketNoi.Open();           //Mở kết nối
        if (ketNoi.State == ConnectionState.Open) //Kiểm tra trạng thái kết nối
            lblKetqua.Text= "ServerVersion: " + ketNoi.ServerVersion + "\nState: " + ketNoi.State;
    }
    catch (Exception)
    {
        if (ketNoi.State != ConnectionState.Open)
            lblKetqua.Text= "Kết nối thất bại";
    }
    finally
    {
        if (ketNoi.State == ConnectionState.Open)
            ketNoi.Close();      //Đóng kết nối
    }
}
```



LTV Công Nghệ .Net (2014) – Module2 - Bài 4. Tổng quan ADO.Net và Kết nối CSDL

19



3. Xử lý kết nối đến CSDL

□ Khai báo và sử dụng chuỗi kết nối trong tập tin cấu hình

- Thêm Reference **System.Configuration** vào project, sau đó bổ sung khai báo namespace này vào cửa sổ code
- Khai báo chuỗi kết nối trong tập tin cấu hình:

```
<connectionStrings>
  <add name="chuoiKN" connectionString="server=(local);
    Database=QLBansach; trusted_connection=yes;" />
</connectionStrings>
```

- Thực hiện kết nối:

```
ConnectionStringSettings boKetNoi =
    ConfigurationManager.ConnectionStrings["chuoiKN"];
string chuoiKetNoi = boKetNoi.ConnectionString;
//Tới đây chúng ta đã có chuỗi kết nối
```



Nội dung



1. Tổng quan về ADO.Net

1. Khái niệm
2. .Net Framework Data Providers

2. Mô hình truy xuất dữ liệu

1. Kết nối
2. Ngắt kết nối

3. Xử lý kết nối đến CSDL SQL Server

4. Minh họa kết nối với SQL Server





4. Minh họa kết nối với SQL Server

❑ Phiên bản là SQL Server Developer

- Xem các ví dụ trước

❑ Phiên bản là SQL Server Express, sử dụng AttachFile

- Ví dụ:



4. Minh họa kết nối với SQL Server



```
string chuoiketnoi = @"Server=(localdb)\V11.0;Integrated Security=true;
AttachDbFileName=|DataDirectory|\DuLieu\QLSinhvien.mdf;";
using (SqlConnection ketnoi = new SqlConnection(chuoiketnoi))
{
    try
    {
        ketnoi.Open();
        if (ketnoi.State == ConnectionState.Open)    lblKq.Text = "Thành công";
    } catch (Exception ex) {
        if (ketnoi.State != ConnectionState.Open)
            lblKq.Text = "Kết nối thất bại " + ex.Message;
    } finally {
        if (ketnoi.State == ConnectionState.Open)    ketnoi.Close();
    }
}
```



Thảo luận





Trường ĐH Khoa Học Tự Nhiên Tp. Hồ Chí Minh
TRUNG TÂM TIN HỌC

Go Screen Capture

LTV CÔNG NGHỆ .NET

Module 2 - Bài 5. *Mô hình kết nối*

Ngành Lập trình & CSDL

<http://www.t3h.vn>

2014



2014





Nội dung

1. Tổng quan về mô hình kết nối – Connected Model

1. Giới thiệu
2. Trình tự thực hiện
3. Đối tượng Command dùng để làm gì?

2. Các Command class trong .Net Framework

3. Tạo và thực hiện Command

4. Sử dụng tham số trong Command



LTV Công Nghệ .Net (2014) – Module2 - Bài 5. Mô hình kết nối

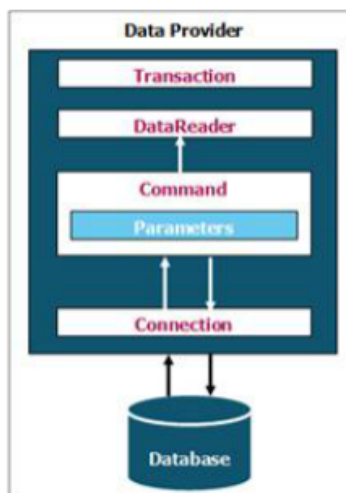
2

1. Tổng quan về mô hình kết nối



□ Giới thiệu

- Mô hình kết nối (Connected Model) cần duy trì một kết nối mở trong suốt quá trình thao tác dữ liệu
- Sử dụng các đối tượng như Connection, Command và DataReader



LTV Công Nghệ .Net (2014) – Module2 - Bài 5. Mô hình kết nối

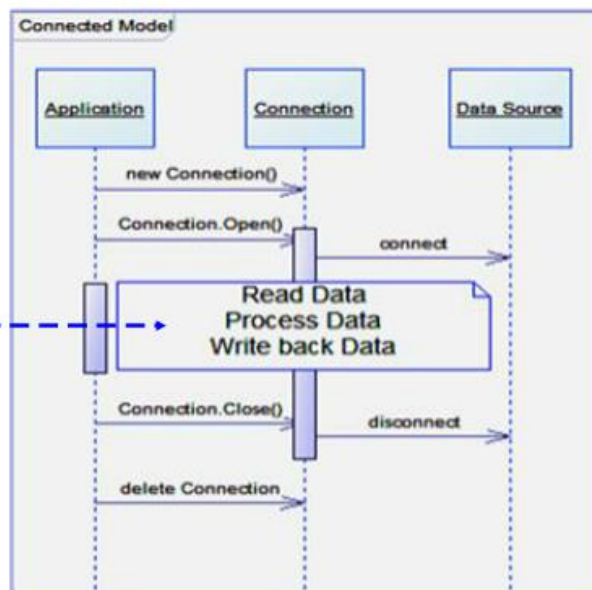
3

1. Tổng quan về mô hình kết nối

□ Trình tự thực hiện

- Khai báo và thực hiện kết nối đến CSDL
- Đọc, xử lý và ghi dữ liệu
- Đóng kết nối với CSDL

Command sẽ thực hiện



LTV Công Nghệ .Net (2014) – Module2 - Bài 5. Mô hình kết nối

4

1. Tổng quan về mô hình kết nối

□ Đối tượng Command dùng để làm gì?

- Thực hiện câu lệnh SQL hoặc thủ tục nội tại (stored procedure), hoặc hàm do người dùng định nghĩa (**U**ser-**D**efined **F**unction – UDF)

LTV Công Nghệ .Net (2014) – Module2 - Bài 5. Mô hình kết nối

5



Nội dung

1. Tổng quan về mô hình kết nối
2. Các Command class trong .Net Framework
3. Tạo và thực hiện Command
4. Sử dụng tham số trong Command



2. Các Command class trong .Net Framework

- ☐ .Net Framework cung cấp các lớp Command tương ứng với các .Net Data Provider
 - System.Data.SqlClient.SqlCommand
 - System.Data.OleDbClient.OleDbCommand
 - System.Data.Odbc.OdbcCommand
 - System.Data.OracleClient.OracleCommand
- ☐ Trong chương trình học, chúng ta sẽ sử dụng SqlCommand class để tạo đối tượng Command





Nội dung

1. Tổng quan về mô hình kết nối
2. Các Command class trong .Net Framework
3. Tạo và thực hiện Command
 1. Khởi tạo đối tượng
 2. Thực hiện Command
 3. Các phương thức thi hành câu lệnh SQL
4. Sử dụng tham số trong Command



3. Tạo và thực hiện Command

□ Khởi tạo đối tượng Command

- Ba thuộc tính cơ bản sau thường được sử dụng để khởi tạo một đối tượng Command
 - **CommandText:**
 - Là câu lệnh SQL (khi CommandType=CommandType.Text)
 - Là tên thủ tục nội tại (khi CommandType = CommandType.StoredProcedure)
 - **CommandType:**
 - CommandType.**Text**: thi hành câu lệnh SQL (giá trị mặc định)
 - CommandType.**StoredProcedure**: thi hành thủ tục nội tại
 - **Connection:** Đối tượng kết nối đến CSDL





3. Tạo và thực hiện Command

□ Khởi tạo đối tượng Command

- Cú pháp 1:

SqlCommand Tên_Command = new SqlCommand();

- Ví dụ 1:

```
//Tạo đối tượng Command
SqlCommand boLenh = new SqlCommand();
//Khai báo thuộc tính
boLenh.CommandText = "Select count(*) From Sach";
boLenh.CommandType = CommandType.Text;
boLenh.Connection = ketNoi;
```



3. Tạo và thực hiện Command

□ Khởi tạo đối tượng Command

- Cú pháp 2:

**SqlCommand Tên_Command = new SqlCommand(Câu_lệnh,
Kết_nối);**

- Ví dụ 2:

```
//Tạo chuỗi lệnh
string lenh = "select count(*) from Sach";
//Tạo đối tượng Command
SqlCommand boLenh = new SqlCommand(lenh, ketNoi);
```





3. Tạo và thực hiện Command

❑ Thực hiện Command

- Phải mở Connection trước khi thực hiện lệnh
- Sử dụng các phương thức ExecuteScalar, ExecuteReader hoặc ExecuteNonQuery để thực hiện câu lệnh
- Phải đóng Connection sau khi thực hiện lệnh
- Ví dụ:

```
ketNoi.Open();
int n = (int)boLenh.ExecuteScalar();
ketNoi.Close();
lblKetqua.Text = "Số sản phẩm : " + n.ToString();
```



3. Tạo và thực hiện Command

❑ Các phương thức thi hành câu lệnh SQL

- **ExecuteScalar**: kết quả trả về là một giá trị
 - Trả về giá trị cột đầu tiên của dòng đầu tiên trong tập kết quả
 - Thi hành các câu truy vấn tính toán thống kê
- Ví dụ 1:

```
//Tạo chuỗi lệnh
string lenh = "Select count(*) from Sach";
//Tạo đối tượng bộ lệnh
SqlCommand boLenh = new SqlCommand(lenh, ketNoi);
//Thực hiện Command
ketNoi.Open();
int n = (int)boLenh.ExecuteScalar();
ketNoi.Close();
```





3. Tạo và thực hiện Command

□ Các phương thức thi hành câu lệnh SQL

- **ExecuteReader**: kết quả trả về là một tập các dòng. Sử dụng đối tượng **DataReader** để lưu kết quả này
 - Thi hành các câu truy vấn rút trích dữ liệu
- Ví dụ 2:

```
//Tạo chuỗi lệnh
string lenh = "Select * from Sach";
//Tạo đối tượng bộ lệnh
SqlCommand boLenh = new SqlCommand(lenh, ketNoi);
//Thực hiện Command
ketNoi.Open();
SqlDataReader ketQua = boLenh.ExecuteReader();
ketNoi.Close();
```



3. Tạo và thực hiện Command

□ Các phương thức thi hành câu lệnh SQL

- **ExecuteNonQuery**: kết quả trả về là số dòng bị ảnh hưởng
 - Thi hành các câu truy vấn cập nhật dữ liệu/ định nghĩa dữ liệu
- Ví dụ 3:

```
//Tạo chuỗi lệnh
string lenh = "Update Sach set GiaBan = GiaBan*1.1";
//Tạo đối tượng bộ lệnh
SqlCommand boLenh = new SqlCommand(lenh, ketNoi);
//Thực hiện Command
ketNoi.Open();
Object ketQua = boLenh.ExecuteNonQuery();
ketNoi.Close();
```





3. Tạo và thực hiện Command

❑ Các phương thức thi hành câu lệnh SQL

- Ví dụ 4: Đọc dữ liệu từ DataReader, sử dụng phương thức **Read()**

```
ConnectionStringSettings boKetNoi = ConfigurationManager.ConnectionStrings["chuoiKN"];
string chuoiKetNoi = boKetNoi.ConnectionString;

SqlConnection ketNoi = new SqlConnection(chuoiKetNoi); //Tạo đối tượng Connection
string lenh = "Select * from ChuDe"; //Tạo chuỗi lệnh
SqlCommand boLenh = new SqlCommand(lenh, ketNoi); //Tạo đối tượng Command
ketNoi.Open(); //Thực hiện Command

SqlDataReader ketQua = boLenh.ExecuteReader();

StringBuilder str = new StringBuilder();

while (ketQua.Read()) { //Duyệt tập kết quả
    for (int i = 0; i < ketQua.FieldCount; i++) //Duyệt tập hợp Field
        str.Append(ketQua[i].ToString() + "; "); //Truy xuất giá trị của Field thứ i
    str.AppendLine(); }

lblKetQua.Text = str.ToString(); //Xuất ra màn hình

ketNoi.Close();
```



LTV Công Nghệ .Net (2014) – Module2 - Bài 5. Mô hình kết nối

16

3. Tạo và thực hiện Command

❑ Các phương thức thi hành câu lệnh SQL

- Ví dụ 5: CommandType là StoredProcedure
 - Đầu tiên, chúng ta tạo StoreProcedure có tên là **spLaySach** với đoạn lệnh tạo như sau:

```
CREATE PROC spLaySach
AS
BEGIN
    SELECT SachID, TenSach FROM Sach Where MaChuDe='CNTTI'
END
```



LTV Công Nghệ .Net (2014) – Module2 - Bài 5. Mô hình kết nối

17



3. Tạo và thực hiện Command

❑ Các phương thức thi hành câu lệnh SQL

• Ví dụ 5: CommandType là StoredProcedure

```
//Tạo đối tượng Connection ketNoi
SqlCommand boLenh= ketNoi.CreateCommand();           //Tạo đối tượng Command
boLenh.CommandType=CommandType.StoredProcedure;
boLenh.CommandText="spLaySach";
ketNoi.Open();                                         //Thực hiện Command
SqlDataReader ketQua = boLenh.ExecuteReader(CommandBehavior.CloseConnection);
StringBuilder str = new StringBuilder();
while (ketQua.Read()) {                               //Duyệt tập kết quả
    for (int i = 0; i < ketQua.FieldCount; i++)         //Duyệt tập hợp Field
        str.Append(ketQua[i].ToString() + "; ");      //Truy xuất giá trị của Field thứ i
    str.AppendLine();
}
lblKetQua.Text = str.ToString();                      //Xuất ra màn hình
ketQua.Close(); //Sẽ tự động đóng Connection, nhờ tham số CommandBehavior.CloseConnection)
```



Nội dung



1. Tổng quan về mô hình kết nối
2. Các Command class trong .Net Framework
3. Tạo và thực hiện Command
4. Sử dụng tham số trong Command
 1. Các Parameter class trong .Net Framework
 2. Khởi tạo đối tượng Parameter
 3. Gắn Parameter vào Command
 4. Thực thi Command
 5. Truy xuất kết quả





4. Sử dụng tham số trong Command

❑ Command có tham số là gì?

- Chuỗi lệnh SQL hoặc StoreProcedure chứa các tham số
- Tham số trong câu lệnh SQL có dạng **@Tên_tham_số**

❑ Khi nào sử dụng tham số trong Command?

- Khi các giá trị của biểu thức điều kiện trong mệnh đề WHERE
 - Khi các giá trị cần cập nhật trong câu lệnh UPDATE, INSERT, DELETE
- Cả hai **chưa được xác định lúc coding**



4. Sử dụng tham số trong Command

❑ Các Parameter class trong .Net Framework

- System.Data.SqlClient.SqlParameter
- System.Data.OleDbClient.OleDbParameter
- System.Data.Odbc.OdbcParameter
- System.Data.OracleClient.OracleParameter





4. Sử dụng tham số trong Command

□ Khởi tạo đối tượng Parameter

- Sử dụng các thuộc tính sau để thực hiện việc khai báo và khởi tạo:
 - ParameterName**: tên tham số
 - DbType**: Kiểu dữ liệu
 - Size**: kích thước
 - Direction**: loại tham số (Input/Output/InputOutput/ReturnValue)
 - Value**: giá trị của tham số



4. Sử dụng tham số trong Command

□ Khởi tạo đối tượng Parameter

- Cú pháp 1:


```
SqlParameter Tên_parameter = new SqlParameter();
```
- Ví dụ 1:

```
//Tạo tham số
SqlParameter pTen = new SqlParameter();
//Khai báo các thuộc tính
pTen.ParameterName = "@tenChuDe";
pTen.SqlDbType = SqlDbType.NVarChar;
pTen.Size = 50;
pTen.Value = "Công nghệ thông tin";
```





4. Sử dụng tham số trong Command

❑ Khởi tạo đối tượng Parameter

- Cú pháp 2:

SqlParameter Tên_parameter = new SqlParameter(Tên_parameter, Kiểu_dữ_liệu);

- Ví dụ 2:

```
//Tạo tham số
SqlParameter pTen = new SqlParameter("@tenChuDe",
SqlDbType.NVarChar);

//Khai báo các thuộc tính
pTen.Size = 50;
pTen.Value = "Công nghệ thông tin";
```



4. Sử dụng tham số trong Command

❑ Gắn Parameter vào Command

- Command có thuộc tính tập hợp **Parameters**
- Thuộc tính Parameters chứa danh sách các tham số của Command
- Sử dụng phương thức **Add** của thuộc tính Parameters để gắn các tham số cho Command
- Cú pháp:

đ.tượng_Command.Parameters.Add(đ.tượng_Parameter);

❑ Thực thi Command

- Code giống Cammand không có tham số

❑ Truy xuất kết quả

- Code giống Cammand không có tham số





4. Sử dụng tham số trong Command

□ Tóm tắt các bước thực hiện Command có tham số

- Tạo đối tượng Connection
- Tạo đối tượng Command
- Khai báo các tham số (Parameter)
- Khởi tạo các thuộc tính cho tham số
- Gắn các tham số cho Command
- Thực hiện Command
- Nhận kết quả ứng với các loại tham số: Loại Input (mặc định), loại đầu ra (Output) hoặc tham số trả về (ReturnValue)



2 bước này là giống với Command không tham số



4. Sử dụng tham số trong Command



□ Ví dụ:

- Tăng giá bán của các sách có Mã Chủ đề và giá tăng được nhập từ bàn phím





4. Sử dụng tham số trong Command

❑ Ví dụ:

- Tăng giá bán của các sách có Mã Chủ đề và giá tăng được nhập từ bàn phím

```
using(SqlConnection ketNoi = new SqlConnection(chuoiKetNoi)) //Đã khai báo chuoiKetNoi trước đó
{
    SqlCommand boLenh = ketNoi.CreateCommand(); //Tạo Command từ đ.tượng Connection
    boLenh.CommandText = "Update Sach Set GiaBan=GiaBan+@tang Where MaChuDe=@ma";
    SqlParameter pGia = new SqlParameter("@tang", SqlDbType.Int);
    pGia.Value = Convert.ToInt32(txtGiaBan.Text);
    SqlParameter pMa = new SqlParameter("@ma", SqlDbType.Char);
    pMa.Value = txtMa.Text;
    boLenh.Parameters.Add(pGia); boLenh.Parameters.Add(pMa);
    ketNoi.Open();
    int n = boLenh.ExecuteNonQuery(); ketNoi.Close();
}
```



Thảo luận





Trường Đại học Khoa Học Tự Nhiên Tp. Hồ Chí Minh
TRUNG TÂM TIN HỌC

LTV CÔNG NGHỆ .NET

Module 2 - Bài 6. *Mô hình ngắt kết nối*

Ngành Lập trình & CSDL

<http://www.t3h.vn>

2014



2014



Nội dung

1. Tổng quan về mô hình ngắt kết nối – Disconnected Model

1. Giới thiệu
2. Trình tự thực hiện

2. Đọc và lưu trữ dữ liệu

3. Cập nhật dữ liệu trong DataTable

4. Cập nhật các thay đổi trong DataTable về CSDL



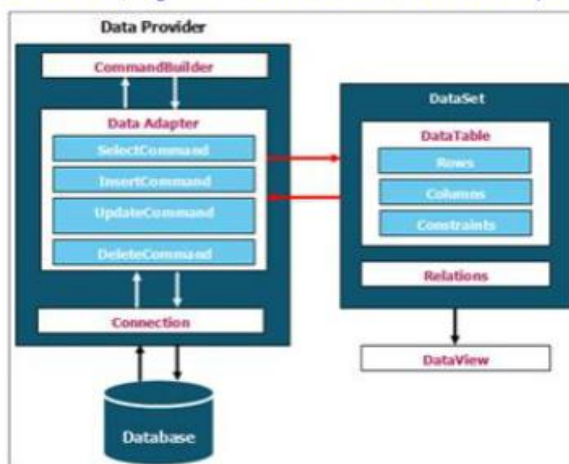
LTV Công Nghệ .Net (2014) – Module2 - Bài 6. Mô hình ngắt kết nối

2

1. Tổng quan về mô hình ngắt kết nối

□ Giới thiệu

- Cho phép người dùng kết nối đến CSDL, đọc và lưu trữ dữ liệu vào vùng nhớ, sau đó ngắt kết nối. Có thể thực hiện các cập nhật trên vùng nhớ này và ghi trở lại CSDL
- Sử dụng các đối tượng như Connection, DataAdapter và DataSet



LTV Công Nghệ .Net (2014) – Module2 - Bài 6. Mô hình ngắt kết nối

3

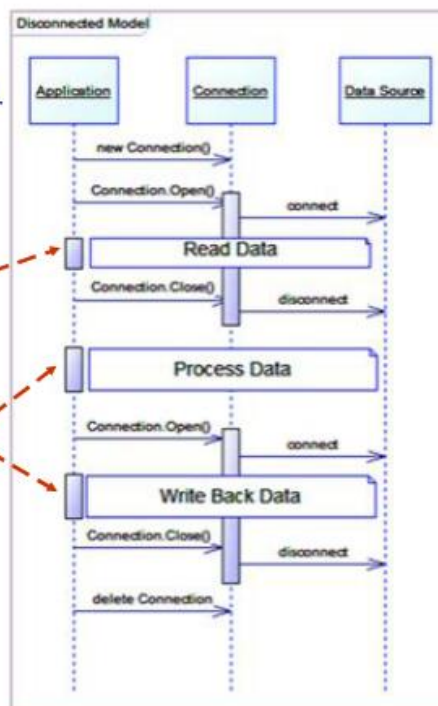
1. Tổng quan về mô hình ngắt kết nối

□ Trình tự thực hiện

- Khai báo và thực hiện kết nối đến CSDL
- Đọc dữ liệu
- Đóng kết nối lại
- Xử lý dữ liệu
- Mở kết nối
- Ghi dữ liệu vào CSDL
- Đóng kết nối với CSDL

DataAdapter

DataSet



LTV Công Nghệ .Net (2014) – Module2 - Bài 6. Mô hình ngắt kết nối

4

Nội dung

1. Tổng quan về mô hình ngắt kết nối – Disconnected Model

2. Đọc và lưu trữ dữ liệu

- Connection
- DataSet
- DataTable
- DataAdapter
- Constraint
- Relation

3. Cập nhật dữ liệu trong DataTable

4. Cập nhật các thay đổi trong DataTable về CSDL



LTV Công Nghệ .Net (2014) – Module2 - Bài 6. Mô hình ngắt kết nối

5



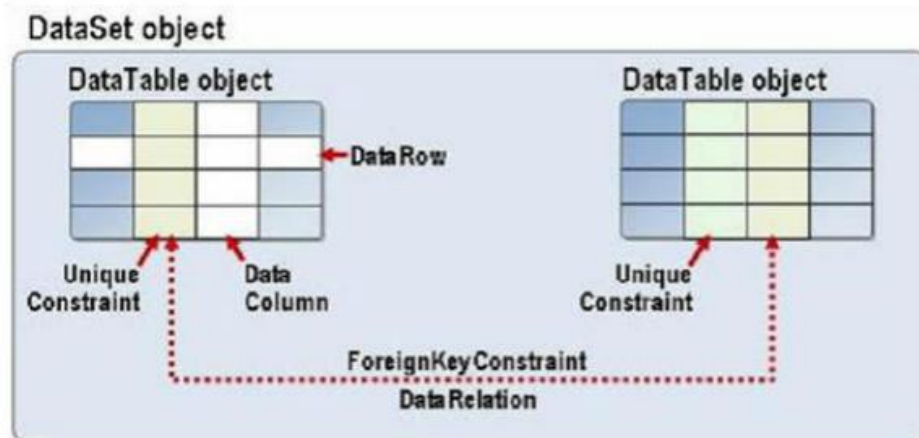
2. Đọc và lưu trữ dữ liệu

- ❑ Xử lý Đọc và Lưu trữ dữ liệu cần những đối tượng: **Connection, DataSet, DataTable, DataAdapter, Constraint, Relation**
- ❑ Đối tượng DataSet
 - DataSet là gì?
 - Là một CSDL thu nhỏ đặt trong vùng nhớ
 - DataSet chứa các đối tượng DataTable và DataRelation
 - Thuộc tính **Tables** chứa các DataTable của DataSet
 - Thuộc tính **Relations** chứa các DataRelation của DataSet
 - Namespace hỗ trợ cho DataSet và các thành phần bên trong nó là **System.Data**



2. Đọc và lưu trữ dữ liệu

- ❑ Đối tượng DataSet
 - Kiến trúc của DataSet

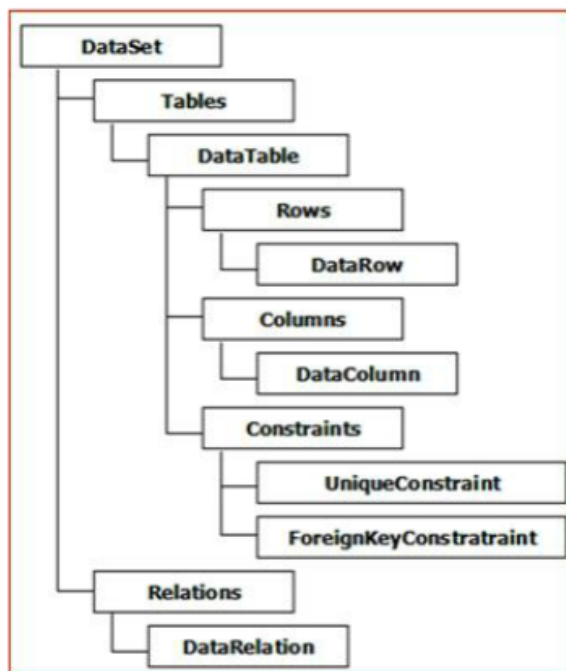




2. Đọc và lưu trữ dữ liệu

❑ Đối tượng DataSet

- Kiến trúc của DataSet



LTV Công Nghệ .Net (2014) – Module2 - Bài 6. Mô hình ngắt kết nối

8

2. Đọc và lưu trữ dữ liệu



❑ Đối tượng DataSet

- Khai báo và khởi tạo DataSet:

//Tạo DataSet (có tên mặc định là NewDataSet)

DataSet Tên_đối_tượng = new DataSet();

// Hoặc tạo DataSet có tên cụ thể

DataSet Tên_đối_tượng = new DataSet("Tên_DataSet");

- Ví dụ:

//tạo DataSet có tên là QLBansach

DataSet duLieu = new DataSet("QLBansach");



LTV Công Nghệ .Net (2014) – Module2 - Bài 6. Mô hình ngắt kết nối

9



2. Đọc và lưu trữ dữ liệu

❑ Đối tượng DataTable

- Là đối tượng đại diện cho các bảng chứa dữ liệu trong DataSet
- Thuộc tính **Columns** chứa các đối tượng DataColumn trong DataTable
- Cấu trúc bên trong:

Columns {

Column Name	Data Type	Allow Nulls
MaChuDe	char(5)	☐
TenChuDe	nvarchar(100)	☒
		☐

DataColumn

- Khai báo và khởi tạo:

//Tạo DataTable không có tên

DataTable Tên_đối_tượng = new DataTable();

// Hoặc tạo DataTable có tên cụ thể

DataTable Tên_đối_tượng = new DataTable("tên_DataTable");



2. Đọc và lưu trữ dữ liệu



❑ Đối tượng DataTable

- Kết DataTable vào DataSet:

Đối_tượng_DataSet.Tables.Add(Đối_tượng_DataTable);

- Ví dụ:

//Tạo DataSet duLieu

DataSet duLieu = new DataSet("QLBansach");

//Tạo DataTable Sach

DataTable bangSach = new DataTable("Sach");

//Kết DataTable Sach vào DataSet duLieu

duLieu.Tables.Add(bangSach);





2. Đọc và lưu trữ dữ liệu

❑ Đối tượng DataAdapter

- Dùng để đọc và ghi dữ liệu từ CSDL, lưu trữ dữ liệu trong các DataTable của DataSet
- Trước khi đọc hoặc ghi thì Connection tự động mở ra và sau khi đọc hoặc ghi xong thì Connection tự động đóng lại
- DataAdapter sử dụng các Command sau:
 - **Select Command:** Nhiệm vụ là đọc dữ liệu
 - **Insert Command:** Nhiệm vụ là thêm dòng dữ liệu
 - **Update Command:** Nhiệm vụ là cập nhật dữ liệu
 - **Delete Command:** Nhiệm vụ là xóa các dòng dữ liệu



2. Đọc và lưu trữ dữ liệu

❑ Đối tượng DataAdapter

- Các class DataAdapter trong .Net Framework
 - `System.Data.SqlClient.SqlDataAdapter`
 - `System.Data.OleDbClient.OleDbDataAdapter`
 - `System.Data.Odbc.OdbcDataAdapter`
 - `System.Data.OracleClient.OracleDataAdapter`
- Khai báo và khởi tạo:

```
SqlDataAdapter boDocGhi = new SqlDataAdapter(chuỗi_lệnh,  
                                             Đối_tượng_connection);
```





2. Đọc và lưu trữ dữ liệu

❑ Đối tượng DataAdapter

• Đọc dữ liệu vào DataSet:

▪ Sử dụng phương thức Fill()

// Fill với tên mặc định

Đ.tượng_DataAdapter.Fill(Đ.tượng_DataSet);

// Fill với tên chỉ định cho các DataTable

Đ.tượng_DataAdapter.Fill(Đ.tượng_DataSet , "Tên_bảng");

▪ Với Fill không chỉ định tên Table thì các DataTable trong DataSet sẽ lần lượt được đặt tên theo quy tắc sau:

- <DataAdapter>.Tables[0]: có tên là "Table"
- Từ DataTable thứ 2 trong DataSet trở đi sẽ có tên là TableN với N tăng dần và bắt đầu từ 1: Table1, Table2, ...



2. Đọc và lưu trữ dữ liệu



❑ Đối tượng DataAdapter

• Đọc dữ liệu vào DataSet:

▪ Ví dụ: Đọc dữ liệu từ bảng ChuDe, sau đó liên kết (binding) dữ liệu vào điều khiển comboBox

//Giả sử đã khai báo chuỗiKetNoi

using (SqlConnection ketNoi = new SqlConnection(chuoiKetNoi))

{

 DataSet duLieu = new DataSet("QLBansach"); *//Tạo DataSet duLieu*

//Đọc dữ liệu từ CSDL vào DataSet và lưu trong bảng Chude

 string chuoiLenh = "Select * from ChuDe";

 SqlDataAdapter boDocGhi = new SqlDataAdapter(chuoiLenh,ketNoi);

 boDocGhi.Fill(duLieu, "Chude");

//Liên kết nguồn dữ liệu với điều khiển comboBox

 cboChude.DataSource = duLieu.Tables["Chude"];

 cboChude.DisplayMember = "TenChuDe";

 cboChude.ValueMember = "MaChuDe";

}





2. Đọc và lưu trữ dữ liệu

❑ Đối tượng DataAdapter

- Đọc dữ liệu vào DataSet:
 - Liên kết (Binding) dữ liệu vào điều khiển comboBox, ListBox và DataGridView, sử dụng thuộc tính:
 - **DataSource**: Nguồn dữ liệu
 - Riêng comboBox và ListBox thì khai báo thêm 2 thuộc tính:
 - **DisplayMember**: Tên thuộc tính thể hiện dữ liệu
 - **ValueMember**: Tên thuộc tính lưu trữ dữ liệu



2. Đọc và lưu trữ dữ liệu

❑ Đối tượng DataAdapter

- Đọc dữ liệu vào DataSet:
 - Ví dụ: Binding dữ liệu vào 3 điều khiển sau

//Liên kết nguồn DL với điều khiển comboBox

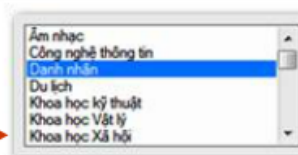
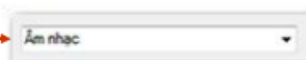
```
cboChude.DataSource = duLieu.Tables["Chude"];
cboChude.DisplayMember = "TenChuDe";
cboChude.ValueMember = "MaChuDe";
```

//Liên kết nguồn DL với điều khiển comboBox và ListBox

```
lstChude.DataSource = duLieu.Tables["Chude"];
lstChude.DisplayMember = "TenChuDe";
lstChude.ValueMember = "MaChuDe";
```

//Liên kết nguồn DL với điều khiển DataGridView

```
dtgChude.DataSource=duLieu.Tables[0];
```



Mã chủ đề	Tên chủ đề
AMNHA	Âm nhạc
CNTTI	Công nghệ thông tin
DNHAN	Danh nhân
DULIC	Du lịch
KHKTH	Khoa học kỹ thuật
KHVLY	Khoa học Vật lý





2. Đọc và lưu trữ dữ liệu

❑ Đối tượng DataAdapter

- Đọc cấu trúc vào DataSet:
 - Nhiều thuộc tính của Table nền sẽ được đưa vào cấu trúc DataTable
 - Sử dụng phương thức `FillSchema()`

Đ.tượng_DataAdapter.FillSchema(Đ.tượng_DataSet, kiểu_cấu_trúc);

- <kiểu cấu trúc>: qui định việc đổi tên có được chấp nhận hay không. Có các giá trị:
 - SchemaType.**Mapped**: sử dụng các TableMappings cho các cấu trúc đưa vào DataSet nếu trùng hợp
 - SchemaType.**Source**: không sử dụng TableMappings



2. Đọc và lưu trữ dữ liệu

❑ Đối tượng DataAdapter

- Đọc cấu trúc vào DataSet:
 - Tùy vào cấu trúc các bảng trên nguồn dữ liệu, những thuộc tính sau sẽ được đưa vào cấu trúc các cột
 - AllowDBNull
 - AutoIncrement
 - MaxLength
 - ReadOnly
 - Unique
 - Chỉ có ràng buộc khóa chính và ràng buộc duy nhất mới được đưa vào Constraints của DataTable





2. Đọc và lưu trữ dữ liệu

❑ Đối tượng DataAdapter

- Đọc cấu trúc vào DataSet:

- Ví dụ: Đọc bảng ChuDe và xem các thuộc tính của DataTable

```
using (SqlConnection ketNoi = new SqlConnection(chuoiKetNoi))
```

```
{
```

```
    DataSet duLieu = new DataSet("QLBansach");
```

```
    string chuoiLenh = "Select * from ChuDe";
```

```
    SqlDataAdapter boDocGhi = new SqlDataAdapter(chuoiLenh, ketNoi);
```

```
    boDocGhi.TableMappings.Add("Table", "CHUDE"); //ánh xạ tên Table vào DataSet
```

```
    boDocGhi.FillSchema(duLieu, SchemaType.Mapped);
```

```
    StringBuilder str = new StringBuilder();
```

```
    str.AppendLine("Khóa chính null? (AllowDBNull) = " + duLieu.Tables[0].Columns[0].AllowDBNull.ToString());
```

```
    str.AppendLine("Tên chủ đề cho phép null? (AllowDBNull) = " + duLieu.Tables[0].Columns[1].AllowDBNull.ToString());
```

```
    str.AppendLine("Chiều dài tên chủ đề (MaxLength) = " + duLieu.Tables[0].Columns[1].MaxLength.ToString());
```

```
    str.AppendLine("Tên Chủ đề là duy nhất? (Unique) = " + duLieu.Tables[0].Columns[1].Unique.ToString());
```

```
    lblKetQua.Text = str.ToString();
```

```
}
```

Khóa chính null? (AllowDBNull) = False
 Tên chủ đề cho phép null? (AllowDBNull) = True
 Chiều dài tên chủ đề (MaxLength) = 100
 Tên Chủ đề là duy nhất? (Unique) = False



2. Đọc và lưu trữ dữ liệu

❑ Đối tượng Constraint

- Constraint là các kiểm tra ràng buộc trên DataTable, có hai loại:
 - **UniqueConstraint** : ràng buộc dữ liệu duy nhất
 - **ForeignKeyConstraint**: ràng buộc khóa ngoại
- Thuộc tính **Constraints** chứa các Constraint của DataTable
- Ví dụ 1: Với ví dụ trước, khi đọc cấu trúc bảng ChuDe thì ràng buộc duy nhất của khóa chính đã được đưa vào

```
duLieu.Tables[0].Columns[0].Unique.ToString(); ➔ true
```

- Và dùng dòng lệnh sau để xem có bao nhiêu ràng buộc trong table này:

```
duLieu.Tables[0].Constraints.Count.ToString(); ➔ 1
```





2. Đọc và lưu trữ dữ liệu

❑ Đối tượng Constraint

- Ví dụ 2: Sau khi đọc cấu trúc bảng ChuDe vào DataSet, ta cũng có thể tạo thêm ràng buộc duy nhất cho Tên Chủ đề

```
//Tạo đối tượng DataColumn từ cột TenChuDe
DataColumn cot = duLieu.Tables[0].Columns["TenChuDe"];
//Tạo ràng buộc duy nhất
UniqueConstraint rangBuoc = new UniqueConstraint("U_TenChude", cot, false);
//Bổ sung ràng buộc vào DataTable
duLieu.Tables[0].Constraints.Add(rangBuoc);
```

- **U_TenChude**: Tên của ràng buộc
- **cot**: đối tượng cột dùng để tạo ràng buộc
- **false (/true)** : false thì không phải là khóa chính, ngược lại là khóa chính



2. Đọc và lưu trữ dữ liệu

❑ Đối tượng Relation

- Relation là quan hệ giữa hai DataTable trong DataSet
- Thuộc tính **Relations** chứa các **DataRelation** của DataSet
- Khi tạo DataRelation, cần phải khai báo: Tên quan hệ, các cột bảng cha, các cột của bảng con
- Cú pháp:

```
DataRelation Đ_tượng = new DataRelation("Tên_relation", cột_cha, cột_con)
```

- Gắn Relation vào DataSet:

```
Đ.tượng_DataSet.Relations.Add( Đ.tượng_DataRelation);
```





2. Đọc và lưu trữ dữ liệu

❑ Đối tượng Relation

- Ví dụ: Đọc 2 bảng ChuDe và Sach vào DataSet, sau đó thiết lập quan hệ giữa 2 bảng này

```
using (SqlConnection ketNoi = new SqlConnection(chuoiKetNoi))
{
    DataSet duLieu = new DataSet("QLBansach");
    string chuoiLenh = "Select * from ChuDe;Select * from Sach";
    SqlDataAdapter boDocGhi = new SqlDataAdapter(chuoiLenh, ketNoi);
    boDocGhi.TableMappings.Add("Table", "CHUDE");
    boDocGhi.TableMappings.Add("Table1", "SACH");
    boDocGhi.FillSchema(duLieu, SchemaType.Mapped);
    boDocGhi.Fill(duLieu);

    ....
}
```



2. Đọc và lưu trữ dữ liệu



❑ Đối tượng Relation

- Ví dụ: Đọc 2 bảng ChuDe và Sach vào DataSet, sau đó thiết lập quan hệ giữa 2 bảng này

```
//tạo quan hệ
if (!duLieu.Relations.Contains("CD_S")) //Kiểm tra đã có relation CD_S chưa?
{
    DataColumn cotCha = duLieu.Tables[0].Columns["MaChuDe"];
    DataColumn cotCon = duLieu.Tables[1].Columns["MaChuDe"];
    DataRelation re = new DataRelation("CD_S", cotCha, cotCon);
    duLieu.Relations.Add(re);
}

//Xuất dữ liệu ra DataGridView
dtgChude.DataSource = duLieu.Tables[0];
dtgSach.DataSource = duLieu.Tables[0];
dtgSach.DataMember = "CD_S";
}
```



Nội dung

1. Tổng quan về mô hình ngắt kết nối – Disconnected Model
2. Đọc và lưu trữ dữ liệu
3. Cập nhật dữ liệu trong DataTable
 1. Đối tượng DataRow
 2. Thêm dữ liệu
 3. Xóa dữ liệu
 4. Ghi dữ liệu
 5. Khôi phục dữ liệu
4. Cập nhật các thay đổi trong DataTable về CSDL



LTV Công Nghệ .Net (2014) – Module2 - Bài 6. Mô hình ngắt kết nối

26

3. Cập nhật dữ liệu trong DataTable

□ Đối tượng DataRow

- Là đối tượng đại diện cho các dòng trong DataTable
- Thuộc tính **Rows** của DataTable chứa các DataRow

	MaChuDe	TenChuDe
▶	AMNHA	Âm nhạc
	CNTTI	Công nghệ thông tin
	DNHAN	Danh nhân
	DULIC	Du lịch
	KHKTH	Khoa học kỹ thuật

- Truy xuất giá trị các cột trong DataRow:

Đối_tượng_DataRow["Tên_cột" | chỉ_số_cột]

- Ví dụ:

```
DataRow dong = bangChude.Rows[0]; // Tạo DataRow
dong["MaChuDe"] = "HOA"; // hoặc dong[0] = "HOA"
```



LTV Công Nghệ .Net (2014) – Module2 - Bài 6. Mô hình ngắt kết nối

27



3. Cập nhật dữ liệu trong DataTable

❑ Thêm dòng dữ liệu

- Tạo mới một DataRow
- Gán các giá trị cho các cột của DataRow
- Sử dụng phương thức **Add** để thêm DataRow vào DataTable
- Ví dụ 1:

```
// Tạo mới DataRow
DataRow dong = bangChude.NewRow();
// Gán giá trị cho các cột
dong["MaChuDe"] = "HOA";
dong["TenChuDe"] = "Các loài hoa";
// Thêm DataRow vào DataTable
bangChude.Rows.Add(dong);
```



3. Cập nhật dữ liệu trong DataTable

❑ Xóa dữ liệu

- Sử dụng các phương thức:
 - **Remove(đối_tượng_DataRow), RemoveAt(chỉ_số_dòng):** xóa hẳn một DataRow ra khỏi DataTable
 - **Delete():** đánh dấu xóa một DataRow trong DataTable
- Ví dụ 2:

```
// Xóa dòng đầu tiên trong bảng bangChude
bangChude.Rows.Remove(bangChude.Rows[0]);    //hoặc
bangChude.Rows.RemoveAt(0);    //hoặc
bangChude.Rows[0].Delete();
```





3. Cập nhật dữ liệu trong DataTable

❑ Ghi dữ liệu

- Sử dụng phương thức **AcceptChanges** để ghi nhận những thay đổi tác động lên các DataTable
- Cú pháp: **Đối_tượng_DataTable.AcceptChanges()**
- Ví dụ 3: sau khi thêm 1 dòng mới trong ví dụ 1, ta đồng ý ghi nhận thay đổi này

```
bangChude.AcceptChanges();
```

- Lưu ý: nếu muốn ghi nhận những thay đổi trên tất cả các DataTable trong DataSet thì cú pháp viết lại như sau:

Đối_tượng_DataSet.AcceptChanges()



3. Cập nhật dữ liệu trong DataTable

❑ Khôi phục dữ liệu

- Sử dụng phương thức **RejectChanges** để không ghi nhận những thay đổi tác động lên các DataTable
- Cú pháp: **Đối_tượng_DataTable|DataSet.RejectChanges()**
- Ví dụ 4: sau khi thêm 1 dòng mới trong ví dụ 1, ta hủy bỏ thay đổi này

```
bangChude.RejectChanges();
```





Nội dung

1. Tổng quan về mô hình ngắt kết nối – Disconnected Model
2. Đọc và lưu trữ dữ liệu
3. Cập nhật dữ liệu trong DataTable
4. Cập nhật các thay đổi trong DataTable về CSDL
 1. Đối tượng Command Builder
 2. Phương thức Update của DataAdapter



4. Cập nhật các thay đổi trong DataTable về CSDL

❑ Đối tượng CommandBuilder

- Dùng để phát sinh bộ lệnh cập nhật cho DataAdapter
- Khởi tạo CommandBuilder:

**SqlCommandBuilder Tên_đối_tượng =
new SqlCommandBuilder(Đ.tượng_DataAdapter);**

- Lưu ý: Khi khởi tạo DataAdapter, câu lệnh SELECT:
 - phải có khóa chính hoặc cột có giá trị duy nhất, nếu không, CommandBuilder sẽ không phát sinh được các câu lệnh Insert, Update, Delete
 - chỉ chứa duy nhất câu lệnh SELECT của bảng cần cập nhật
 - gồm nhiều câu lệnh SELECT, CommandBuilder chỉ tạo bộ lệnh cập nhật cho câu truy vấn đầu tiên





4. Cập nhật các thay đổi trong DataTable về CSDL

❑ Phương thức Update của DataAdapter

- Dùng để ghi các thay đổi từ DataSet, DataTable về CSDL
- Cú pháp:

```
// Cập nhật các thay đổi trên DataSet về cơ sở dữ liệu
Đối_tượng_DataAdapter.Update( Đối_tượng_DataSet );
// Cập nhật các thay đổi trên DataTable về cơ sở dữ liệu
Đối_tượng_DataAdapter.Update( Đối_tượng_DataTable );
```



4. Cập nhật các thay đổi trong DataTable về CSDL

- ### ❑ Ví dụ: Xây dựng màn hình Thêm mới một Chủ đề, cập nhật về CSDL

Mã chủ đề	Tên chủ đề
VHNNG	Văn học nước ngoài
VHDXHO	Văn hóa - Xã hội
VNCCA	Vật nuôi - Cây cảnh
YHDTO	Y Học dân tộc
HOA	Hoa các loại

Mã chủ đề: HOA

Tên chủ đề: Hoa các loại

Thêm vào CSDL





4. Cập nhật các thay đổi trong DataTable về CSDL

□ **Ví dụ:** Xây dựng màn hình Thêm mới một Chủ đề, cập nhật về CSDL

- Khai báo chung trong cửa sổ code của form:

```
ConnectionStringSettings boKetNoi =
ConfigurationManager.ConnectionStrings["chuoiKN"];
string chuoiKetNoi;
SqlConnection ketNoi;
DataSet duLieu;
SqlDataAdapter boDocGhi;
```



4. Cập nhật các thay đổi trong DataTable về CSDL

□ **Ví dụ:** Xây dựng màn hình Thêm mới một Chủ đề, cập nhật về CSDL

- Xử lý khi load form:

```
try
{
    chuoiKetNoi = boKetNoi.ConnectionString;
    ketNoi = new SqlConnection(chuoiKetNoi);
    duLieu = new DataSet("QLBansach");
    string chuoiLenh = "Select * from ChuDe";
    boDocGhi = new SqlDataAdapter(chuoiLenh, ketNoi);
    boDocGhi.FillSchema(duLieu, SchemaType.Mapped);
    boDocGhi.Fill(duLieu);
    dtgChude.DataSource = duLieu.Tables[0];
} catch (Exception ex)
{
    MessageBox.Show("Kết nối dữ liệu có lỗi: " + ex.Message);
    this.Close();
}
```





4. Cập nhật các thay đổi trong DataTable về CSDL

❑ **Ví dụ:** Xây dựng màn hình Thêm mới một Chủ đề, cập nhật về CSDL

- Xử lý nút **Thêm vào CSDL:**

```
try
{
    SqlCommandBuilder boPhatSinh = new SqlCommandBuilder(boDocGhi); //Phát sinh bộ lệnh
    DataTable bang = duLieu.Tables[0];
    DataRow dong = bang.NewRow(); //Tạo dòng mới
    dong["MaChuDe"] = txtMa.Text;    dong["TenChuDe"] = txtTen.Text; //Gán giá trị
    bang.Rows.Add(dong); //Thêm dòng mới
    boDocGhi.Update(duLieu); //Ghi vào CSDL
    MessageBox.Show("Thêm dữ liệu thành công");
} catch (Exception ex) {
    MessageBox.Show("Quá trình cập nhật bị lỗi: " + ex);
}
```



Thảo luận





Trường ĐH Khoa Học Tự Nhiên Tp. Hồ Chí Minh
TRUNG TÂM TIN HỌC

LTV CÔNG NGHỆ .NET

Module 2 - Bài 7. *XML cơ bản*

Ngành Lập trình & CSDL

<http://www.t3h.vn>

2014



2014





Nội dung

1. Tổng quan về XML

1. Khái niệm
2. Tài liệu XML
3. Cấu trúc tài liệu XML
4. Ứng dụng của XML trong Visual Studio

2. Đọc, ghi XML trong DataSet

1. Tổng quan
2. Tạo tài liệu XML trong Visual Studio
3. Đọc tài liệu XML vào DataSet
4. Ghi DataSet vào tài liệu XML



1. Tổng quan về XML

□ Khái niệm

- XML- EXtensible Markup Language
- Là một ngôn ngữ đánh dấu giống HTML
- Được dùng để trao đổi thông tin chứ không phải hiển thị dữ liệu giống như HTML
- XML là một trong các tiêu chuẩn của W3C





1. Tổng quan về XML

❑ Tài liệu XML

- Là văn bản có cấu trúc theo **cú pháp XML**, cho phép biểu diễn thông tin về các đối tượng trong thực tế
- Ví dụ:
 - Đối tượng x thuộc loại X trong thực tế
→ Thẻ X trong tài liệu Xml
 - Thuộc tính a của đối tượng x thuộc loại X trong thực tế
→ Thuộc tính a của thẻ X trong tài liệu Xml

Nhân viên Phạm Thiên Thanh trong thực tế, thẻ biểu diễn như sau:

```
<NhanVien HoTen="Phạm Thiên Thanh" />
```

//<NhanVien> là thẻ

// HoTen là thuộc tính

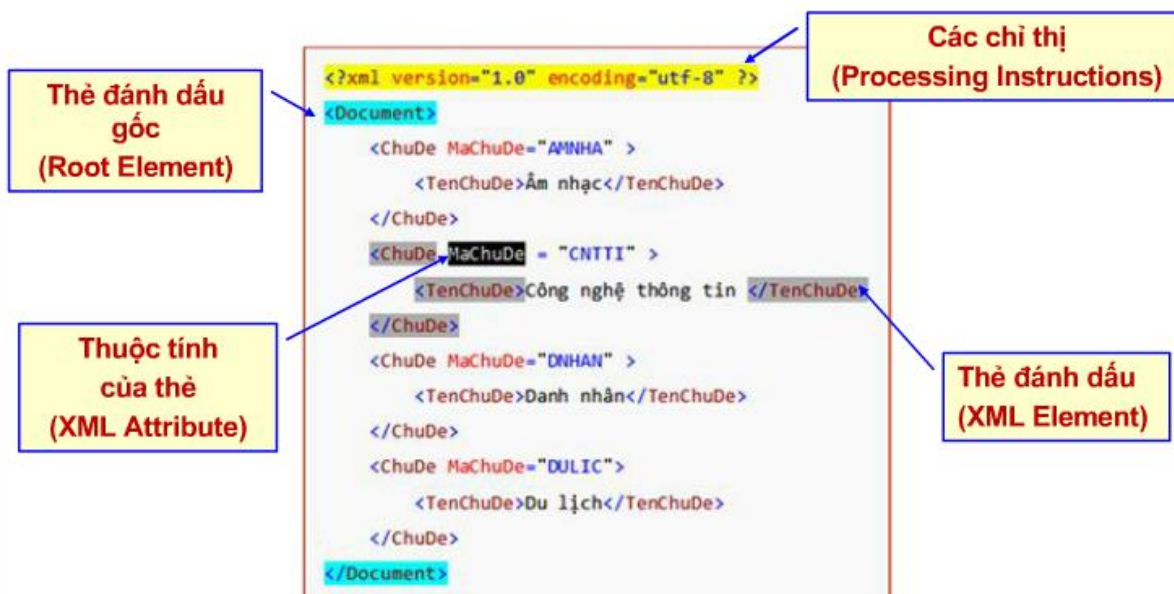


LTV Công Nghệ .Net (2014) – Module2 - Bài 7. XML cơ bản

4

1. Tổng quan về XML

❑ Cấu trúc của tài liệu XML



LTV Công Nghệ .Net (2014) – Module2 - Bài 7. XML cơ bản

5



1. Tổng quan về XML

□ Cấu trúc của tài liệu XML

- **Các chỉ thị** (Processing Instructions) nằm ở đầu tài liệu xml và nằm trong cặp thẻ **<? và ?>**
 - Thông thường, các chỉ thị này giúp bộ phân tích (parser) tài liệu xml biết các thông tin:
 - **Version:** phiên bản xml
 - **encoding:** bộ mã mà tài liệu XML sử dụng, thường có giá trị là utf-8.
- **Thẻ đánh dấu gốc** (Root element): Tài liệu XML phải có duy nhất (một và chỉ một) thẻ chứa (trực tiếp hay gián tiếp) tất cả các thẻ còn lại

```
<?xml version="1.0" encoding="utf-8" ?>
<Document>
    ...
</Document>
```



1. Tổng quan về XML



□ Cấu trúc của tài liệu XML

- **Thẻ đánh dấu** (XML element): Các thẻ đánh dấu trong tài liệu XML chỉ bao gồm 2 loại:
 - **Thẻ có nội dung**, có dạng: **<Tên_thẻ> Nội dung </Tên_thẻ>**
 - **Thẻ rỗng**, có dạng: **<Tên_thẻ [Tên_thuộc_tính="giá_trị"] />**
 - **Tên thẻ:**
 - Có thể chứa các ký tự, số, ...
 - Không được bắt đầu bằng số hay ký tự đặc biệt
 - Không được bắt đầu bằng chuỗi xml hay XML hay Xml ...
 - Có phân biệt chữ hoa, chữ thường (Case Sensitive)
 - Được phép lồng các thẻ vào nhau, nhưng có trình tự





1. Tổng quan về XML

□ Cấu trúc của tài liệu XML

• Thẻ đánh dấu (XML element):

- Ví dụ: biểu diễn phân số 3/4 có 2 cách

```
<?xml version="1.0" encoding="utf-8"?>
<PhanSo>
    <TuSo>3</TuSo>
    <MauSo>4</MauSo>
</PhanSo>
```

```
<?xml version="1.0" encoding="utf-8"?>
<PhanSo TuSo="3" MauSo="4" />
```



1. Tổng quan về XML

□ Cấu trúc của tài liệu XML

• Thuộc tính của thẻ (XML Attribute):

<Tên_thẻ Tên_thuộc_tính = "Giá_trị" >

- Giá_trị của thuộc tính phải đặt trong cặp nháy "

(Xem ví dụ trong slide trước)

• Tạo ghi chú (comment) trong tài liệu XML

<!-- nội dung ghi chú -->





1. Tổng quan về XML

□ Ứng dụng của XML trong Visual Studio

- Lưu trữ thông tin cấu hình của ứng dụng trong các file: Web.config, App.config, ..
- Mô tả dữ liệu của Web Services và cách thức truy xuất
- Trao đổi thông tin thông qua các file tài liệu XML, có 2 cách:
 - Dùng code (sử dụng mô hình DOM) trực tiếp để truy xuất dữ liệu từ tài liệu XML hoặc ghi các thay đổi từ ứng dụng vào tài liệu XML:
 - Sử dụng công nghệ ADO.Net để đọc, ghi dữ liệu từ XML vào DataSet hoặc ngược lại
- ...



Nội dung



1. Tổng quan về XML

1. Khái niệm
2. Tài liệu XML
3. Cấu trúc tài liệu XML
4. Ứng dụng của XML trong Visual Studio

2. Đọc, ghi XML trong DataSet

1. Tổng quan
2. Tạo tài liệu XML trong Visual Studio
3. Đọc tài liệu XML vào DataSet
4. Ghi DataSet vào tài liệu XML



2. Đọc, ghi XML trong DataSet

□ Tổng quan

- Trong .NET, ta có thể truy xuất tài liệu XML thông qua mô hình DOM hoặc DataSet
- Mô hình DOM là mô hình chuẩn, đa số các ngôn ngữ lập trình đều hỗ trợ
- Đối với các tài liệu XML lưu dữ liệu dạng bảng giống với CSDL thì ta có thể dùng DataSet để đọc hay ghi dữ liệu dễ dàng



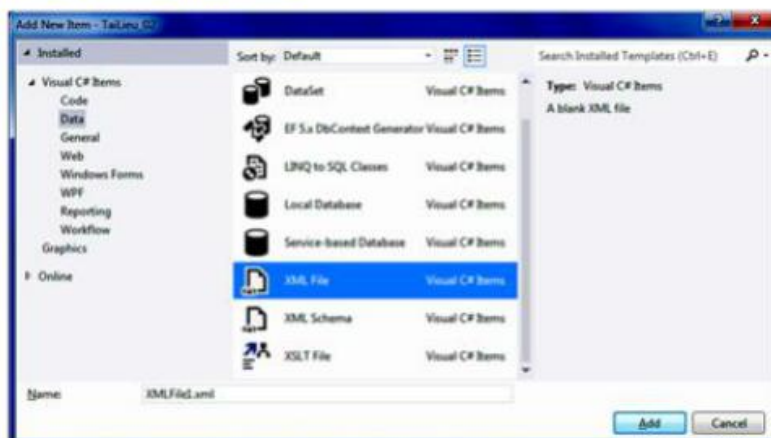
LTV Công Nghệ .Net (2014) – Module2 - Bài 7. XML cơ bản

12

2. Đọc, ghi XML trong DataSet

□ Tạo tài liệu XML trong Visual Studio

- Nhấn phím phải chuột lên project: chọn
 - Add → New Item ...
 - Chọn tiếp như hình sau:



LTV Công Nghệ .Net (2014) – Module2 - Bài 7. XML cơ bản

13



2. Đọc, ghi XML trong DataSet

❑ Đọc tài liệu XML vào DataSet

- Tạo đối tượng DataSet

```
DataSet dt_DataSet = new DataSet();
```

- Đọc tài liệu XML

- `dt_DataSet.ReadXml("đường dẫn file Xml" [, XmlReadMode])`
 - XmlReadMode: xác định cách đọc dữ liệu XML và Schema liên quan
- `dt_DataSet.ReadXmlSchema("đường dẫn file Xml")`: đọc cấu trúc của DataSet mà không cần load dữ liệu



2. Đọc, ghi XML trong DataSet



❑ Đọc tài liệu XML vào DataSet

- Ví dụ: Đọc tài liệu ChuDe.xml vào DataSet, xuất dữ liệu ra DataGridView và ListBox
- Viết code cho nút **Đọc tài liệu XML**



- Khai báo đối tượng DataSet **ds** dùng chung trong form:

```
DataSet ds = new DataSet();
```





2. Đọc, ghi XML trong DataSet

❑ Đọc tài liệu XML vào DataSet

- Ví dụ: Đọc tài liệu ChuDe.xml vào DataSet, xuất dữ liệu ra DataGridView và ListBox

//code của nút Đọc tài liệu XML

```
ds.ReadXml(Environment.CurrentDirectory+ @"..\..\DuLieu\ChuDe.xml", XmlReadMode.Auto);
```

//Xuất ra DataGridView

```
dtgLuoi.DataSource = ds.Tables[0];
```

//Xuất ra ListBox

```
string muc = "";
```

```
foreach (DataRow dong in ds.Tables[0].Rows)
```

```
{
```

```
    muc = dong["MaChuDe"] + ":\t" + dong["TenChuDe"];
```

```
    lstChude.Items.Add(muc);
```

```
}
```



2. Đọc, ghi XML trong DataSet

❑ Ghi DataSet vào tài liệu XML

- Đặt tên cho DataSet (tên nút Gốc của tài liệu XML)

```
dt_DataSet.DataSetName = "Tên_DataSet";
```

- Đặt tên cho các DataTable trong DataSet (tên các nút dòng dữ liệu)

```
dt_DataSet.Tables[0].TableName = "Tên_DataTable";
```

- Ghi DataSet ra file XML

- `dt_DataSet.WriteXml("đường dẫn file Xml" [, XmlWriteMode])`

- XmlWriteMode: có ghi inline schema kèm theo dữ liệu không

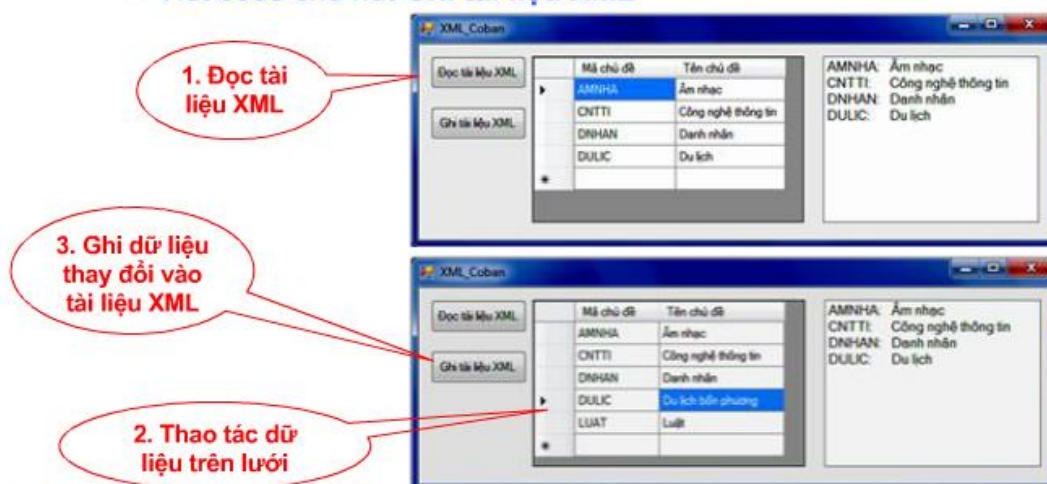
- `dt_DataSet.WriteXmlSchema("đường dẫn file Xml")`: ghi cấu trúc DataSet ra XML schema



2. Đọc, ghi XML trong DataSet

❑ Ghi DataSet vào tài liệu XML

- Ví dụ: Ghi dữ liệu từ DataSet ra tài liệu ChuDe2.xml. Sau khi người dùng thao tác trên lưới thì dữ liệu sẽ thay đổi
 - Viết code cho nút **Ghi tài liệu XML**



LTV Công Nghệ .Net (2014) – Module2 - Bài 7. XML cơ bản

18

2. Đọc, ghi XML trong DataSet

❑ Ghi DataSet vào tài liệu XML

- Ví dụ: Ghi dữ liệu từ DataSet ra tài liệu ChuDe2.xml. Sau khi người dùng thao tác trên lưới thì dữ liệu sẽ thay đổi

```
try
{
    ds.DataSetName = "bangChuDe";
    ds.Tables[0].TableName = "ChuDe";
    ds.WriteXml(Environment.CurrentDirectory + @"..\..\DuLieu\ChuDe2.xml",
        XmlWriteMode.IgnoreSchema);
    MessageBox.Show("Thành công");
}
catch (Exception ex)
{
    MessageBox.Show("Có lỗi: " + ex.Message);
}
```

LTV Công Nghệ .Net (2014) – Module2 - Bài 7. XML cơ bản

19

2. Đọc, ghi XML trong DataSet

❑ Ghi DataSet vào tài liệu XML

- Ví dụ: Ghi dữ liệu từ DataSet ra tài liệu ChuDe2.xml. Sau khi người dùng thao tác trên lưới thì dữ liệu sẽ thay đổi

Chude.xml

```
<?xml version="1.0" encoding="utf-8" ?>
<Document>
  <ChuDe MaChuDe="AMNHA" >
    <TenChuDe>Âm nhạc</TenChuDe>
  </ChuDe>
  <ChuDe MaChuDe="CNTTI" >
    <TenChuDe>Công nghệ thông tin</TenChuDe>
  </ChuDe>
  <ChuDe MaChuDe="DNHAN" >
    <TenChuDe>Đanh nhân</TenChuDe>
  </ChuDe>
  <ChuDe MaChuDe="DULIC">
    <TenChuDe>Du lịch</TenChuDe>
  </ChuDe>
</Document>
```

Chude2.xml

```
<?xml version="1.0" standalone="yes"?>
<bangChuDe>
  <ChuDe MaChuDe="AMNHA">
    <TenChuDe>Âm nhạc</TenChuDe>
  </ChuDe>
  <ChuDe MaChuDe="CNTTI">
    <TenChuDe>Công nghệ thông tin</TenChuDe>
  </ChuDe>
  <ChuDe MaChuDe="DNHAN">
    <TenChuDe>Đanh nhân</TenChuDe>
  </ChuDe>
  <ChuDe MaChuDe="DULIC">
    <TenChuDe>Du lịch bốn phương</TenChuDe>
  </ChuDe>
  <ChuDe MaChuDe="LUAT">
    <TenChuDe>Luật</TenChuDe>
  </ChuDe>
</bangChuDe>
```



LTV Công Nghệ .Net (2014) – Module2 – Bài 7. XML cơ bản

20

Thảo luận



LTV Công Nghệ .Net (2014) – Module2 – Bài 7. XML cơ bản

21



Trường ĐH Khoa Học Tự Nhiên Tp. Hồ Chí Minh
TRUNG TÂM TIN HỌC

LTV CÔNG NGHỆ .NET

Module 2 - Bài 8. *LINQ nâng cao*

Ngành Lập trình & CSDL

<http://www.t3h.vn>

2014



2014





Nội dung

1. Ôn lại LINQ cơ bản

2. Biểu thức Lambda – Lambda Expression

1. Giới thiệu
2. Cú pháp
3. Giới thiệu delegate Func
4. Giới thiệu delegate Action

3. Phương thức mở rộng – Extension Methods

1. Khái niệm
2. Giới thiệu các phương thức mở rộng do .Net cấp



1. Ôn lại LINQ cơ bản

- ❑ Cú pháp LINQ
- ❑ Các thành phần và hoạt động của LINQ
- ❑ Hai cú pháp sử dụng LINQ:

- Query Expression Syntax
- Method Syntax

```
int[] mangSo = { 50, 42, 16, 3, 9, 8, 12, 7, 24, 0 };
// Query Expression syntax
IEnumerable<int> query = from n in mangSo where n <= 3 select n;
// Method syntax
IEnumerable<int> query = mangSo.Where(n => n <= 3)
```

.Where và .Select: gọi là
p.thức mở rộng

.Select(n => n);

n=> n<=3 và n=>n: gọi
là biểu thức Lambda





Nội dung

1. Ôn lại LINQ cơ bản

2. Biểu thức Lambda – Lambda Expression

1. Giới thiệu
2. Cú pháp
3. Giới thiệu delegate Func
4. Giới thiệu delegate Action

3. Phương thức mở rộng – Extension Methods

1. Khái niệm
2. Giới thiệu các phương thức mở rộng do .Net cấp



LTV Công Nghệ .Net (2014) – Module2 - Bài 8. LINQ nâng cao

4

2. Biểu thức Lambda – Lambda Expression



□ Nguồn gốc của biểu thức Lambda

-- ** Sử dụng từ khóa **delegate** để khai báo một mẫu hàm đại diện: 3 cách ** --

//Giả sử đoạn code sau được viết trong cửa sổ code behind của 1 form

```
public delegate double Divide(int param1, int param2);
private void Form1_Load(object sender, EventArgs e)
{
```

//1. Khai báo mẫu hàm đại diện

//cách 1: dùng Named Method

Divide divide1 = new Divide(**LocalDivide**);

//cách 2: dùng Anonymous Method

Divide divide2 = **delegate**(int param1, int param2)

```
{
    return (double)param1 / param2;
};
```

//cách 3: dùng Lambdas (được giới thiệu từ C#3.0)

Divide divide3 = (**param1, param2**) => (double)param1 / param2;

//2. Sử dụng hàm đã khai báo

double so1= divide1(6, 2); → 3

double so2 = divide2(6, 3); → 2

double so3 = divide3(9, 2); → 4.5

```
}
```

```
double LocalDivide(int p1, int p2)
{
    return (double)p1 / p2;
}
```



LTV Công Nghệ .Net (2014) – Module2 - Bài 8. LINQ nâng cao

5



2. Biểu thức Lambda – Lambda Expression

□ Giới thiệu

- Lambda Expression là 1 inline delegate
- Đó là cách viết ngắn gọn để biểu diễn anonymous method
- Khác với anonymous method, Lambda Expression không yêu cầu kiểu tường minh (kiểu tham số và kiểu trả về), compiler sẽ xác định kiểu của biến dựa trên ngữ cảnh



2. Biểu thức Lambda – Lambda Expression

□ Cú pháp

Tham_số_đầu_vào => Biểu_thức hoặc Khối_lệnh

- **=>** : gọi là toán tử Lambda (Lambda operator)
- **Tham_số_đầu_vào**: Có thể có 1 hoặc nhiều tham số (có kiểu tường minh hoặc không), cách nhau bằng dấu phẩy





2. Biểu thức Lambda – Lambda Expression

□ Cú pháp

• Ví dụ:

```
delegate int del(int i);
private void btnTinh_Click(object sender, EventArgs e)
{
    //Dùng Biểu thức
    del myDelegate = x => x * x;
    int j = myDelegate(5);    //khi gặp dòng này thì nó sẽ gọi biểu thức của delegate trên
    lblKetQua1.Text = j.ToString();    //j = 25
    //Dùng khối lệnh
    del myDel = x =>
    {
        string s = x.ToString(); x += 2;
        lblKetQua2.Text = string.Concat(s, "\n" + x.ToString());
        return x;
    };
    int k = myDel(5);    //khi gặp dòng này thì nó sẽ gọi đoạn code của delegate trên
}
```



LTV Công Nghệ .Net (2014) – Module2 - Bài 8. LINQ nâng cao

8

2. Biểu thức Lambda – Lambda Expression



□ Giới thiệu delegate Func

- **Func<>** : là một họ của nhóm generic delegate đa năng trong **System.Linq**, mục đích là hỗ trợ cách dùng deletage ngắn hơn
- Có 17 versions
 - **Func<TResult>**
 - **Func<T1, ... T16, TResult>**

Trong đó:

- **T1, ... T16**: kiểu tham số truyền vào
- **TResult**: kiểu kết quả trả về
- Khi ta sử dụng **Func<int, bool>**, thì phải hiểu đây là cách viết gọn của **public delegate bool Func(int A);**, trong đó:
 - **int** là kiểu của tham số đầu vào
 - **bool** là kiểu của kết quả trả về



LTV Công Nghệ .Net (2014) – Module2 - Bài 8. LINQ nâng cao

9



2. Biểu thức Lambda – Lambda Expression

□ Giới thiệu delegate Func

• Ví dụ:

```
// khởi tạo với anonymous method
Func<int, bool> del1 =
    delegate(int A)
    {
        return A % 2 == 0;
    };

// Khởi tạo với lambda expression, không cần định nghĩa delegate
Func<int, bool> del2 = A => A % 2 == 0;

// Sử dụng
bool kq1 = del1(5);    //False
bool kq2 = del2(4);    //True
```



LTV Công Nghệ .Net (2014) – Module2 - Bài 8. LINQ nâng cao

10



2. Biểu thức Lambda – Lambda Expression

□ Giới thiệu delegate Action

- Action<> : Giống Func nhưng không trả về giá trị
- Có 16 versions:
 - Action<T1, ... ,T16>

Trong đó:

- T1, ... T16: kiểu tham số truyền vào

• Ví dụ:

```
// Khởi tạo với lambda expression
StringBuilder stb = new StringBuilder();
Action<string> Xuat = s => stb.Append(s);
Xuat("Hoa hồng ");
Xuat("Nhưng");
lblKetQua.Text = stb.ToString();    // Hoa hồng Nhưng
```



LTV Công Nghệ .Net (2014) – Module2 - Bài 8. LINQ nâng cao

11



Nội dung

1. Ôn lại LINQ cơ bản
2. Biểu thức Lambda – Lambda Expression
 1. Giới thiệu
 2. Cú pháp
 3. Giới thiệu delegate Func
 4. Giới thiệu delegate Action
3. Phương thức mở rộng – Extension Methods
 1. Khái niệm
 2. Giới thiệu các phương thức mở rộng do .Net cấp



LTV Công Nghệ .Net (2014) – Module2 - Bài 8. LINQ nâng cao

12

3. Phương thức mở rộng – Extension Method



- Khái niệm
 - Extension methods là các phương thức tĩnh (static) được kết hợp với một kiểu của .Net
 - Tính năng này giúp ta bổ sung các phương thức mới vào các kiểu hiện có mà không cần cập nhật chúng
 - Hỗ trợ truyền delegate function trong đó có Lambda expression
 - Phương thức mở rộng có dạng:

Tên_phương_thức<[danh_sách_tham_số] >



LTV Công Nghệ .Net (2014) – Module2 - Bài 8. LINQ nâng cao

13



3. Phương thức mở rộng – Extension Method

□ Khái niệm

• Ví dụ:

```
var query = from c in dsSach
            where c.MaChuDe = 5
            select new {c.SachID, c.TenSach};

//sẽ được viết lại bằng phương thức mở rộng như sau:
var query = dsSach.Where (c => c.MaChuDe = 5)
                  .Select (c => new {c.SachID, c.TenSach} );

//Trong đó dsSach là một collection có kiểu class Sach
```



3. Phương thức mở rộng – Extension Method

□ Giới thiệu các phương thức do .Net cấp

- Tham số của các phương thức này thường có kiểu Func
- Khi sử dụng các phương mở rộng do .Net cấp, ta phải:
 - Khai báo namespace **System.Linq**
 - Hiểu rõ 2 khái niệm sau đây trong cú pháp truy vấn LINQ:
 - **Element** : Đại diện cho phần tử trong một tập hợp
 - **Sequence** : Đại diện cho một hợp (Array, Collection, Table, ...)





3. Phương thức mở rộng – Extension Method

□ Giới thiệu các phương thức do .Net cấp

• Ví dụ:

```
var query = from c in dsSach
            where c.MaChuDe = 5
            select new {c.SachID, c.TenSach};

//sẽ được viết lại bằng phương thức mở rộng như sau:
var query = dsSach.Where(c => c.MaChuDe = 5)
                  .Select(c => new {c.SachID, c.TenSach});

//Trong đó dsSach là một collection có kiểu class Sach
```

- **dsSach** là Sequence
- **c** là Element



3. Phương thức mở rộng – Extension Method

□ Giới thiệu các phương thức do .Net cấp

- Bảng hướng dẫn sử dụng các phương thức do .Net cấp được trình bày chi tiết trong tài liệu tham khảo

• Ví dụ:

```
int[] dsSo = { 5, 4, 1, 3, 9, 4, 6, 5, 2, 1 };
List<string> dsChuoai = new List<string>{"Thiên", "căn", "ở", "tại", "lòng", "ta", "chữ",
    "Tâm", "kia", "mới", "bằng", "ba", "chữ", "Tài"};

//lọc các số chẵn trong dsSo
IEnumerable<int> query = dsSo.Where(x => x % 2 == 0);

//Lọc các chuỗi có chiều dài >=4
IEnumerable<string> query=dsChuoai.Where(x => x.Length>= 4);

//lọc các chuỗi bắt đầu là chữ T
var query = dsChuoai.Where(x=> x.StartsWith("T"));
```



Thảo luận





Trường ĐH Khoa Học Tự Nhiên Tp. Hồ Chí Minh
TRUNG TÂM TIN HỌC

LTV CÔNG NGHỆ .NET

Module 2 - Bài 9. *Entity Framework*

Ngành Lập trình & CSDL

<http://www.t3h.vn>

2014



2014





Những vấn đề trước đây và hiện nay

- ❑ **Kỹ năng cần thiết đối với người phát triển 1 ứng dụng có truy xuất đến CSDL quan hệ:**
 - Biết ít nhất 2 ngôn ngữ (C# và SQL)
 - Cũng phải biết công nghệ ADO.Net
- ❑ **Dẫn đến vấn đề:**
 - Người lập trình gặp phải các rào cản giữa 2 ngôn ngữ, đó là:



- Cú pháp khác nhau
- Kiểu dữ liệu khác nhau (string ~ varchar)
- Công cụ sử dụng khác nhau
- Mô hình khác nhau: Object và Procedural

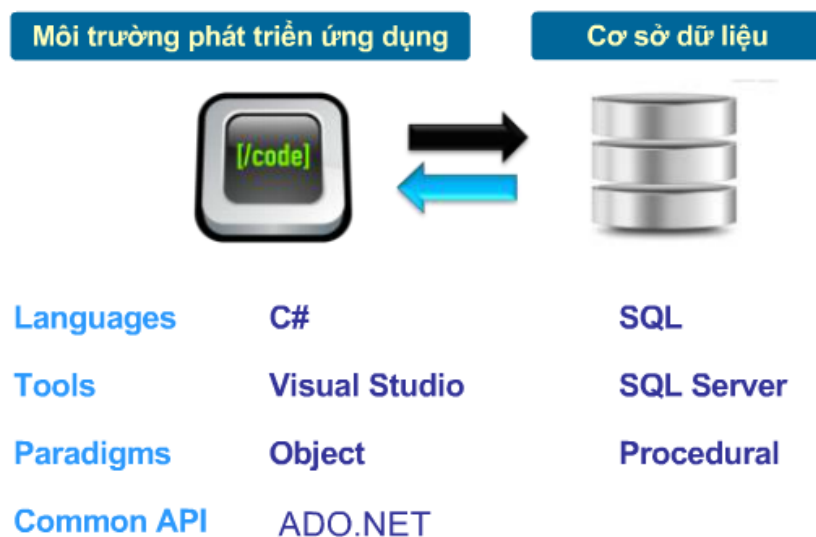


LTV Công Nghệ .Net (2014) – Module2 - Bài 9. Entity Framework

2



Những vấn đề trước đây và hiện nay



LTV Công Nghệ .Net (2014) – Module2 - Bài 9. Entity Framework

3

Những vấn đề trước đây và hiện nay

```
using (SqlConnection ket_noi = new SqlConnection("....."))
{
    ket_noi.Open();
    SqlCommand bo_lenh = ket_noi.CreateCommand();
    bo_lenh.CommandText = "Select * From sinh_vien Where Ma_khoa = @Ma";

    bo_lenh.Parameters.AddWithValue("@Ma", "CN");

    SqlDataReader bo_doc = bo_lenh.ExecuteReader();
    StringBuilder strb = new StringBuilder();
    while (bo_doc.Read())
    {
        string name = bo_doc.GetString(0);
        strb.AppendLine(name);
    }
}
```

1. SQL Query in a string

2. Loosely bound paramters

3. Loosely typed columns

Tiềm ẩn nhiều lỗi vì trình biên dịch không phát hiện được & tốn thời gian

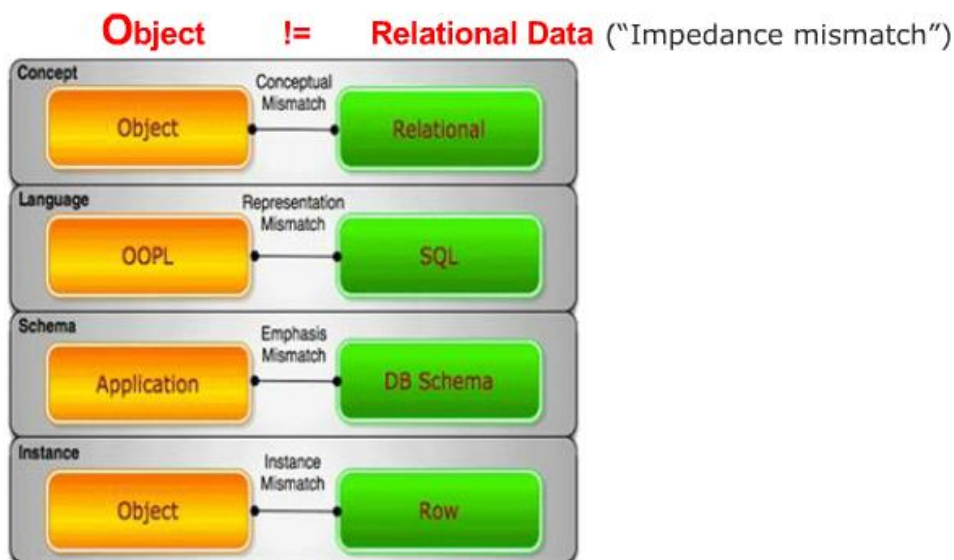


LTV Công Nghệ .Net (2014) – Module2 - Bài 9. Entity Framework

4

Những vấn đề trước đây và hiện nay

- ❑ Tóm lại, vấn đề chính là: sự khác biệt giữa **Object** và **Relational Data** trong một ứng dụng OOP có truy xuất dữ liệu

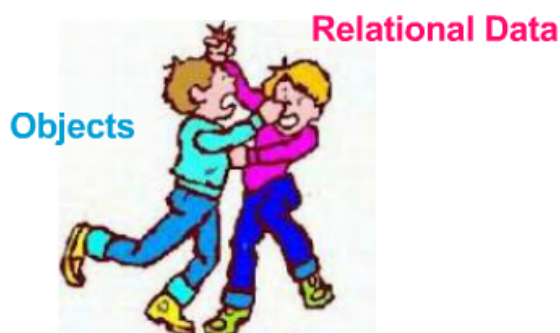


LTV Công Nghệ .Net (2014) – Module2 - Bài 9. Entity Framework

5

Những vấn đề trước đây và hiện nay

- ❑ Trong thời gian qua thì khoảng cách này được thu hẹp bằng cách:
 - Sử dụng ADO.Net kết hợp với lập trình OOP để xây dựng ứng dụng theo kiến trúc n-Tier, hoặc
 - Sử dụng framework của đối tác thứ 3
- nhưng
- Nó tốn **chi phí và thời gian**, vẫn có **lỗi tiềm ẩn**, ...



LTV Công Nghệ .Net (2014) – Module2 - Bài 9. Entity Framework

6

Nội dung

1. Tổng quan về Entity Framework
 1. Entity framework là gì?
 2. Đặc điểm
2. Lịch sử phát triển Entity Framework
3. Kiến trúc Entity Framework
4. Truy vấn và cập nhật dữ liệu trong Entity Framework
5. Các chiến lược tải dữ liệu trong Entity Framework



LTV Công Nghệ .Net (2014) – Module2 - Bài 9. Entity Framework

7



1. Tổng quan về Entity Framework

□ Entity Framework là gì?

- Là phương pháp truy xuất dữ liệu, giúp thu hẹp khoảng cách giữa *data structures* và *objects*
- Là một tập hợp các công nghệ trong ADO.NET
 - Hỗ trợ các ứng dụng hướng dữ liệu (*data-oriented*).
- Dữ liệu được biểu diễn dưới hình thức các đối tượng và thuộc tính cụ thể
 - Không cần phải quan tâm đến việc dữ liệu được lưu trữ trong bảng nào, cột nào trong CSDL



1. Tổng quan về Entity Framework

□ Đặc điểm

- Hỗ trợ các CSDL như: SQL Server, Oracle, MySQL, SQLite, DB2, SQL Azure
- Truy cập dữ liệu bằng phương pháp lập trình dựa trên mô hình quan niệm (*conceptual model*)
- Thoát khỏi sự phụ thuộc mã “chết” (*hard code*) của một dữ liệu cụ thể hoặc lược đồ lưu trữ
- Hỗ trợ truy vấn tích hợp vào ngôn ngữ (LINQ)
 - Lúc biên dịch, kiểm tra cú pháp các truy vấn dựa trên mô hình quan niệm



Nội dung

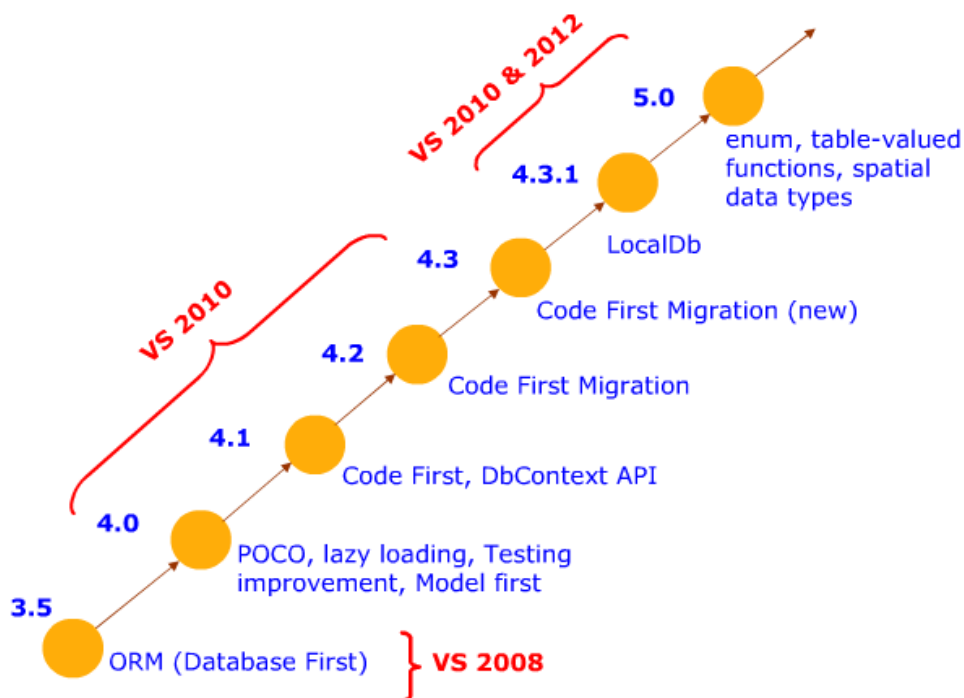
1. Tổng quan về Entity Framework
2. Lịch sử phát triển Entity Framework
3. Kiến trúc Entity Framework
4. Truy vấn và cập nhật dữ liệu trong Entity Framework
5. Các chiến lược tải dữ liệu trong Entity Framework



LTV Công Nghệ .Net (2014) – Module2 - Bài 9. Entity Framework

10

2. Lịch sử phát triển Entity Farmework



LTV Công Nghệ .Net (2014) – Module2 - Bài 9. Entity Framework

11

Nội dung

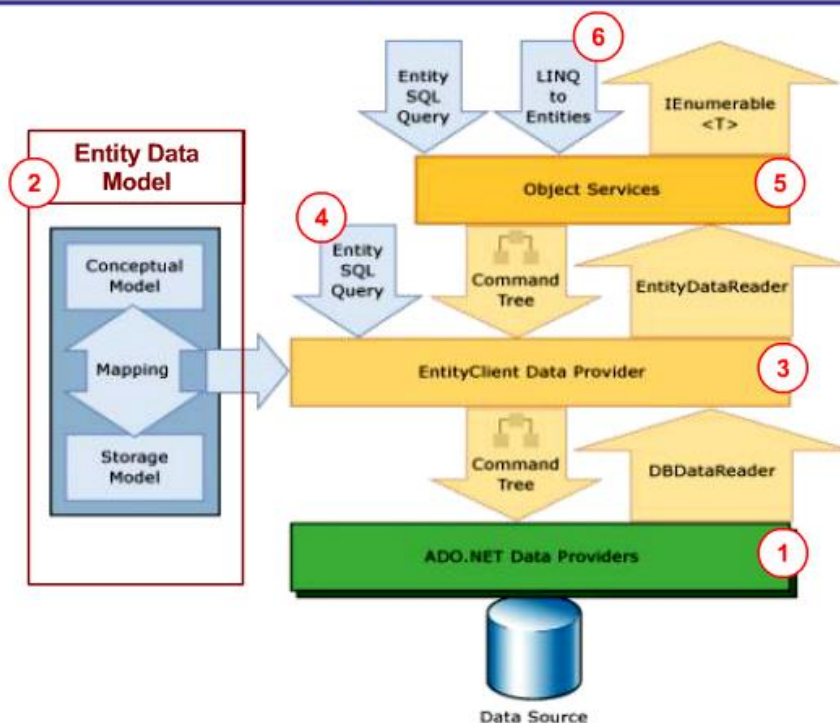
1. Tổng quan về Entity Framework
2. Lịch sử phát triển Entity Framework
3. Kiến trúc Entity Framework
 1. ADO.Net Providers
 2. Entity Data Model
 3. Entity Client Data Provider
 4. Entity SQL (eSQL)
 5. Object Services
 6. LINQ to Entities
4. Truy vấn và cập nhật dữ liệu trong Entity Framework
5. Các chiến lược tải dữ liệu trong Entity Framework



LTV Công Nghệ .Net (2014) – Module2 - Bài 9. Entity Framework

12

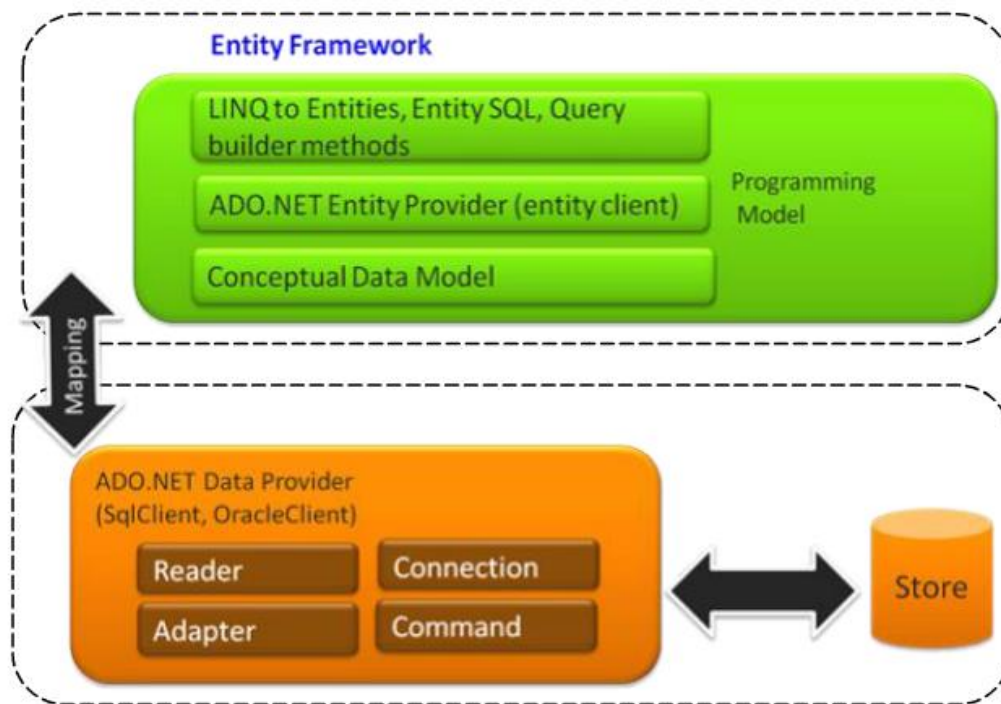
3. Kiến trúc Entity Framework



LTV Công Nghệ .Net (2014) – Module2 - Bài 9. Entity Framework

13

3. Kiến trúc Entity Farmework



LTV Công Nghệ .Net (2014) – Module2 - Bài 9. Entity Framework

14

3. Kiến trúc Entity Farmework

❑ ADO.Net providers

- Layer này sử dụng chuẩn ADO.Net để làm việc với CSDL
- Thi hành command và trả kết quả dựa trên DBMS cụ thể

❑ Entity Data Model

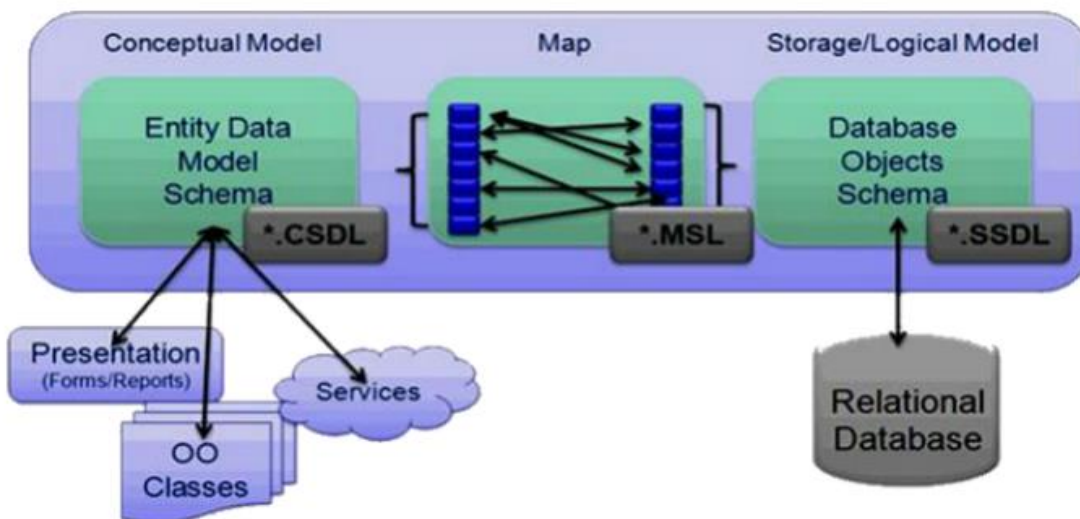
- Là một tập hợp các object mô tả cấu trúc các nghiệp vụ (business data) và ánh xạ đến thành phần lưu trữ bên dưới
- Gồm có 3 layers:
 - **Conceptual**-Conceptual Schema Definition Language (CSDL): ngôn ngữ định nghĩa các entity, relationship
 - **Mapping**-Mapping specification language (MSL): định nghĩa các ánh xạ giữa lớp conceptual và logical
 - **Logical/Storage**-Store Schema Definition Language (SSDL): định nghĩa mô hình lưu trữ của dữ liệu

LTV Công Nghệ .Net (2014) – Module2 - Bài 9. Entity Framework

15

3. Kiến trúc Entity Framework

Entity Data Model



LTV Công Nghệ .Net (2014) – Module2 - Bài 9. Entity Framework

16

3. Kiến trúc Entity Farmework

Entity Data Model

```
<?xml version="1.0" encoding="utf-8"?>
<edmx:Edmx Version="3.0" xmlns:edmx="http://schemas.microsoft.com/ado/2009/11/edmx">
  <!-- EF Runtime content -->
  <edmx:Runtime>
    <!-- SSDL content -->
    <edmx:StorageModels>
      <Schema Namespace="QLBanSachModel." Alias="Self" Provider="System.Data.SqlClient" ProviderAssembly="System.Data.SqlClient" />
    <!-- CSDL content -->
    <edmx:ConceptualModels>
      <Schema Namespace="QLBanSachModel" Alias="Self" p1:UseStrongSpatialTypes="false" />
    </edmx:ConceptualModels>
    <!-- C-S mapping content -->
    <edmx:Mappings>...</edmx:Mappings>
  </edmx:Runtime>
  <!-- EF Designer content (DO NOT EDIT MANUALLY BELOW HERE) -->
  <Designer xmlns="http://schemas.microsoft.com/ado/2009/11/edmx/Designer">...</Designer>
</edmx:Edmx>
```

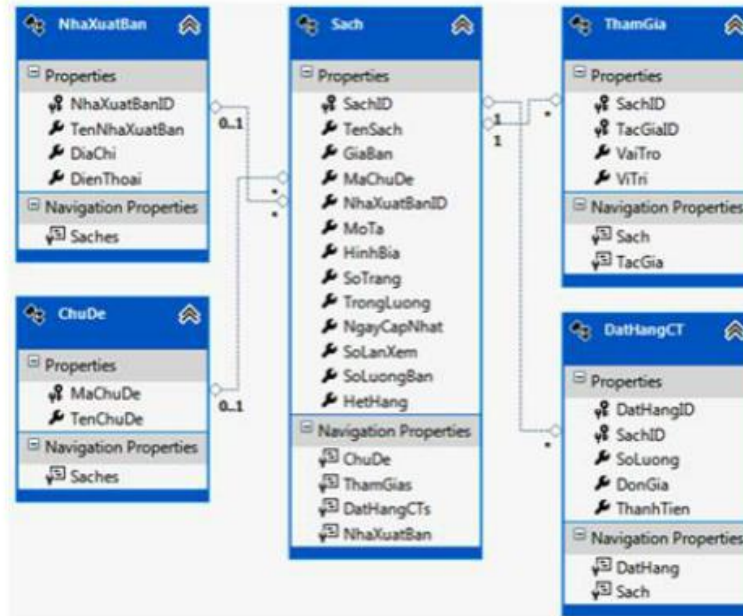


LTV Công Nghệ .Net (2014) – Module2 - Bài 9. Entity Framework

17

3. Kiến trúc Entity Farmework

Entity Data Model



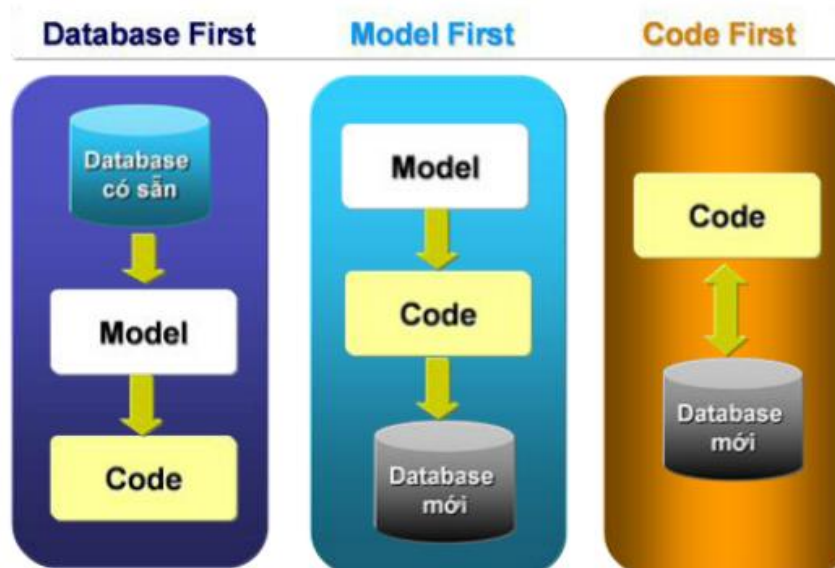
LTV Công Nghệ .Net (2014) – Module2 - Bài 9. Entity Framework

18

3. Kiến trúc Entity Farmework

Entity Data Model

- Có 3 phương pháp tạo EDM



LTV Công Nghệ .Net (2014) – Module2 - Bài 9. Entity Framework

19



3. Kiến trúc Entity Framework

❑ Entity Client Data Provider

- Là một Data Provider mới dùng để truy xuất dữ liệu được mô tả trong mô hình quan niệm
- Các lớp hỗ trợ nằm trong namespace **System.Data.EntityClient**, bao gồm: **EntityConnection**, **EntityCommand**, **EntityDataReader**, ...
- Có các tính năng:
 - Quản lý kết nối (Managing Connections):
 - Sử dụng **EntityConnection**
 - Tạo và thi hành truy vấn (Creating and executing queries)
 - Sử dụng **EntityCommand**, **EntityDataReader**, **EntityParameter**
 - Quản lý giao tác (Managing Transactions)



3. Kiến trúc Entity Framework

❑ Entity SQL (eSQL)

- Là một ngôn ngữ truy vấn lưu trữ độc lập, nó tương tự như SQL
- Cho phép truy vấn thực thể dữ liệu (entity data)
- Lưu ý: ngôn ngữ này truy vấn dựa trên EDM chứ không phải CSDL

❑ Object Services

- Cho phép truy vấn, thêm, cập nhật và xóa dữ liệu
- Biểu diễn dữ liệu dưới dạng các đối tượng có định kiểu mạnh trong CLR
- Hỗ trợ ngôn ngữ **LINQ to Entities** và **eSQL** dựa trên các kiểu được định nghĩa trong mô hình quan niệm
- Cụ thể hóa dữ liệu trả về là các đối tượng, và lưu lại các trạng thái thay đổi về CSDL
- Các lớp hỗ trợ nằm trong **System.Data.Entity**



3. Kiến trúc Entity Farmework

❑ Object Services

- Các class tự động phát sinh tương ứng với mô hình:
 - DbContext: đại diện cho một database, quản lý các kết nối
 - DbSet<TEntity>: là một tập hợp các entity. Mỗi đối tượng này tương ứng với một table

```
public partial class QLBanSachEntities : DbContext
{
    public QLBanSachEntities()
        : base("name=QLBanSachEntities")
    {
    }
    protected override void OnModelCreating(DbModelBuilder modelBuilder)
    {
        throw new UnintentionalCodeFirstException();
    }
    public DbSet<ChuDe> ChuDes { get; set; }
    public DbSet<DatHang> DatHangs { get; set; }
    public DbSet<DatHangCT> DatHangCTs { get; set; }
    public DbSet<KhachHang> KhachHangs { get; set; }
    public DbSet<NhaXuatBan> NhaXuatBans { get; set; }
    public DbSet<Sach> Saches { get; set; }
    public DbSet<TacGia> TacGias { get; set; }
    public DbSet<ThamGia> ThamGias { get; set; }
}
```



LTV Công Nghệ .Net (2014) – Module2 - Bài 9. Entity Framework

22

3. Kiến trúc Entity Farmework

❑ LINQ to Enitties

- Là phương pháp truy vấn dữ liệu thông qua mô hình thực thể dữ liệu (Entity Data Model – EDM) mà EF hỗ trợ
- Chuyển đổi truy vấn LINQ thành truy vấn câu lệnh (Command tree queries)



LTV Công Nghệ .Net (2014) – Module2 - Bài 9. Entity Framework

23



Nội dung

1. Tổng quan về Entity Framework
2. Lịch sử phát triển Entity Framework
3. Kiến trúc Entity Framework
4. Truy vấn và cập nhật dữ liệu trong Entity Framework
 1. Sử dụng LINQ to Entities (LTE)
 1. Truy vấn dữ liệu
 2. Cập nhật dữ liệu
5. Các chiến lược tải dữ liệu trong Entity Framework



4. Truy vấn và cập nhật dữ liệu trong Farmework

- ☐ Sử dụng eSQL để thực hiện truy vấn LINQ
 - Học viên sẽ được học trong Module 3
- ☐ Sử dụng LINQ to Entity để
 - Truy vấn dữ liệu
 - Cập nhật dữ liệu





4. Truy vấn và cập nhật dữ liệu trong Farmework

❑ Truy vấn dữ liệu

- Bước 1: Khai báo và khởi tạo đối tượng **DbContext**

```
DbContext Tên_context = new
DbContext("Tên_db_hoặc_chuỗi_Connection");
```

Hoặc, dùng strong-typed context (context có định kiểu)

```
Context_định_kiểu Tên_context = new Context_định_kiểu();
```

- Ví dụ:

```
DbContext db = new DbContext("name=QLBanSachEntities");
//Hoặc dùng strong-typed Context
QLBanSachEntities db2 = new QLBanSachEntities();
```



4. Truy vấn và cập nhật dữ liệu trong Farmework

❑ Truy vấn dữ liệu

- Bước 2: Tạo câu truy vấn
 - Sử dụng đối tượng **DbSet<TEntity>**, hoặc phương thức **Set<TEntity>** (trả về **DbSet**)
 - Kết hợp với các phương thức mở rộng để viết các câu truy vấn





4. Truy vấn và cập nhật dữ liệu trong Farmework

❑ Truy vấn dữ liệu

• Ví dụ:

```
DbContext db = new DbContext("name=QLBanSachEntities");
var query = db.Set<NhaXuatBan>();
//hoặc
var query = db.Set<NhaXuatBan>().Where(x=>x.TenNhaXuatBan.StartsWith("T"));
dtgLuoì.DataSource = query.ToList(); //Lưu ý: phải chuyển sang List khi binding
//hoặc dùng strong-typed context
QLBanSachEntities db2 = new QLBanSachEntities();
DbSet<ChuDe> dschuDe = db2.ChuDes;
dtgLuoì.DataSource = dschuDe.ToList();
//Ví dụ : trả về kiểu vô danh
var query = db.KhachHangs.Select(x => new {x.KhachHangID, x.HoTen, x.DiaChi });
```



4. Truy vấn và cập nhật dữ liệu trong Farmework

❑ Cập nhật dữ liệu

• Thêm mới:

```
Đ.tượng_DbContext.đ.tượng_DbSet.Add(đ.tượng_thực_thể)
//Hoặc
Đ.tượng_DbContext.Entry(đ.tượng_thực_thể).State =
System.Data.Entity.EntityState.Added
```

▪ Ví dụ:

```
using (QLBanSachEntities db = new QLBanSachEntities())
{
    ChuDe entity = new ChuDe { MaChuDe = "HOA", TenChuDe = "Hoa các loại" };
    db.ChuDes.Add(entity); //hoặc
    db.Entry(entity).State = System.Data.Entity.EntityState.Added;
}
```





4. Truy vấn và cập nhật dữ liệu trong Farmework

□ Cập nhật dữ liệu

• Thêm mới:

- Thuộc tính **State**: trạng thái của entity, gồm các giá trị:
 - **Added**: Đang được bổ sung vào context
 - **Unchanged**: Các giá trị trong context chưa thay đổi
 - **Modified**: Các giá trị trong context đã thay đổi nhưng chưa cập nhật vào CSDL
 - **Deleted**: Đang được đánh dấu xóa trong context
 - **Detached**: Thực thể không nằm trong context hiện hành (nó không theo dõi)



4. Truy vấn và cập nhật dữ liệu trong Farmework

□ Cập nhật dữ liệu

• Xóa:

Đ.tượng_DbContext.đ.tượng_DbSet.Remove(đ.tượng_thực_thể)

//Hoặc xóa nhiều entity

Đ.tượng_DbContext.đ.tượng_DbSet.RemoveRange(Tập_thực_thể)

- Ví dụ 1: xóa 1 dòng dữ liệu có Mã chủ đề là “HOA”

```
using (QLBanSachEntities db = new QLBanSachEntities())
{
    ChuDe entity = db.ChuDes.FirstOrDefault(x => x.MaChuDe == "HOA");
    if (entity != null)
        db.ChuDes.Remove(entity);
}
```





4. Truy vấn và cập nhật dữ liệu trong Farmework

❑ Cập nhật dữ liệu

• Xóa:

- Ví dụ 2: xóa nhiều dòng dữ liệu có Mã chủ đề bắt đầu là "Z"

```
using (QLBanSachEntities db = new QLBanSachEntities())
{
    var entitys = db.ChuDes.FirstOrDefault(x => x.MaChuDe.StartsWith("Z"));
    if (entitys != null)
        db.ChuDes.RemoveRange(entitys);
}
```



4. Truy vấn và cập nhật dữ liệu trong Farmework

❑ Cập nhật dữ liệu

• Ghi xuống CSDL:

Đ. tượng DbContext.SaveChanges()

- Tất cả thay đổi trên context sẽ thật sự ghi xuống CSDL khi ta dùng cú pháp này
- Ví dụ: sau khi hiệu chỉnh 1 entity có sẵn, ta cập nhật xuống CSDL

```
try
{
    ChuDe entity = db.ChuDes.FirstOrDefault(x => x.MaChuDe == txtMaChude.Text.Trim());
    if (entity != null)
    {
        entity.TenChuDe = txtTenChude.Text.Trim();
        db.SaveChanges();
    }
} catch (EntityException ex) { MessageBox.Show(ex.Message); }
```



Nội dung

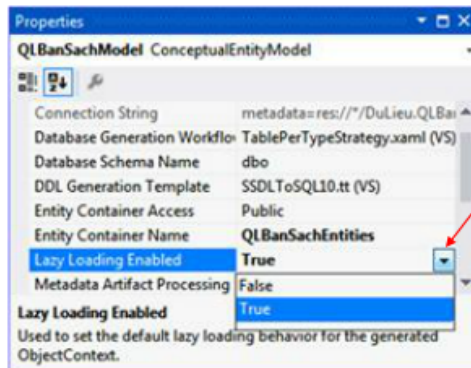
1. Tổng quan về Entity Framework
2. Lịch sử phát triển Entity Framework
3. Kiến trúc Entity Framework
4. Truy vấn và cập nhật dữ liệu trong Entity Framework
5. Các chiến lược tải dữ liệu trong Entity Framework
 1. Lazy/Deferred loading
 2. Eager loading
 3. Explicit loading



5. Các chiến lược tải dữ liệu trong EF

□ Lazy/Deferred Loading

- Lazy loading là quá trình mà một hoặc một tập thực thể được load tự động từ cơ sở dữ liệu lần đầu tiên hay không. Nếu tắt nó thì thuộc tính tham chiếu đến thực thể (Navigation properties) được truy xuất (nghĩa là truy xuất các thực thể có liên quan)
- **Thao tác:** Mở cửa sổ EDM, nhấn phím phải chuột, chọn **Properties**



Hiệu chỉnh xong, nhớ lưu lại





5. Các chiến lược tải dữ liệu trong EF

□ Lazy/Deferred Loading

- Ví dụ 1: vào cửa sổ code của EDM ta, khai báo như sau:

```
public QLBanSachEntities() : base("name=QLBanSachEntities")
{
    this.Configuration.LazyLoadingEnabled = false;
}
//khai báo đoạn code sử dụng như sau sẽ không bị lỗi:
using (QLBanSachEntities dbc = new QLBanSachEntities())
{
    var query = dbc.ChuDes;
    dtgLuoi.DataSource = query.ToList();
}
//Nhưng nếu sửa lại giá trị trên là true thì nó sẽ bị lỗi vì không load được tập
thực thể Saches có liên quan
```



5. Các chiến lược tải dữ liệu trong EF



□ Eager Loading

- Lúc Lazy Loading bị tắt, thì khi tạo câu truy vấn ta phải dùng Eager hoặc Explicit loading để load các thực thể có liên quan
- Ngược lại Lazy, Eager loading là quá trình load những thực thể có liên quan đến thực thể đang truy vấn khi được yêu cầu từ người dùng
- Sử dụng phương thức **Include** để thực hiện Eager Loading
- Trong ứng dụng web, chúng ta thường bật Lazy Loading để tăng hiệu quả đọc dữ liệu
 - Lúc nào cần dữ liệu liên quan thì ta dùng Include





5. Các chiến lược tải dữ liệu trong EF

❑ Eager Loading

- Ví dụ 2: sửa lại ví dụ 1, dùng Eager loading

```
public QLBanSachEntities() : base("name=QLBanSachEntities")
{
    this.Configuration.LazyLoadingEnabled = true;
}
//khai báo đoạn code sử dụng như sau sẽ không bị lỗi:
using (QLBanSachEntities dbc = new QLBanSachEntities())
{
    var query = dbc.ChuDes.
        .Include(b => b.Saches);    // hoặc .Include("Saches")
    dtgLuoi.DataSource = query.ToList();
}
```



5. Các chiến lược tải dữ liệu trong EF



❑ Explicit Loading

- Gọi là load tường minh
- Sử dụng phương thức **Load()**
- Ví dụ:

```
using (QLBanSachEntities dbc = new QLBanSachEntities())
{
    ChuDe entity = dbc.ChuDes.Find("CNTTI");    //tìm thực thể với giá trị là khóa chính
    if (entity != null)
    {
        dbc.Entry(entity).Collection(x => x.Saches).Load();
        var query = entity.Saches.Select( x=> new {x.MaChuDe, x.TenSach });
        dtgLuoi.DataSource = query.ToList();
    }
}
```



Thảo luận

